

Network Computing

(058172)



POLITECNICO
MILANO 1863

Software-Defined Networking

The Google example



In the last lecture we concluded with the Google example. In particular:

- Google adopts centralized control protocols

The Google example



In the last lecture we concluded with the Google example. In particular:

- Google adopts centralized control protocols
 - Control and management becomes substantially more complex with Clos topologies as the number of switching elements increases.

The Google example



In the last lecture we concluded with the Google example. In particular:

- Google adopts centralized control protocols
 - Control and management becomes substantially more complex with Clos topologies as the number of switching elements increases.
 - Existing routing and management protocols were not well-suited to such an environment

The Google example



In the last lecture we concluded with the Google example. In particular:

- Google adopts centralized control protocols
 - Control and management becomes substantially more complex with Clos topologies as the number of switching elements increases.
 - Existing routing and management protocols were not well-suited to such an environment
 - Idea: global configuration of predefined cluster plan proactively pushed on all switches

Content

- Legacy Router/Switching architecture
- Innovation in the Network Control plane with Software-Defined Networking
- Innovation in the Network Data plane (just a brief stint)

Legacy router/switch architecture

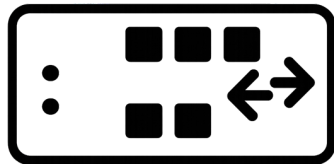
What's a switch?

Legacy router/switch architecture

What's a switch?

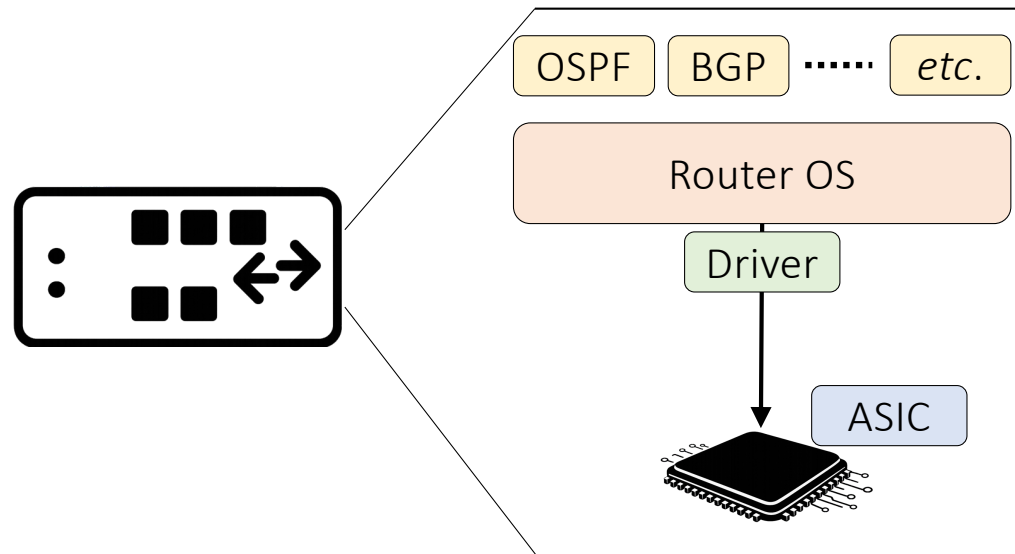
What's a router?

Legacy router/switch architecture

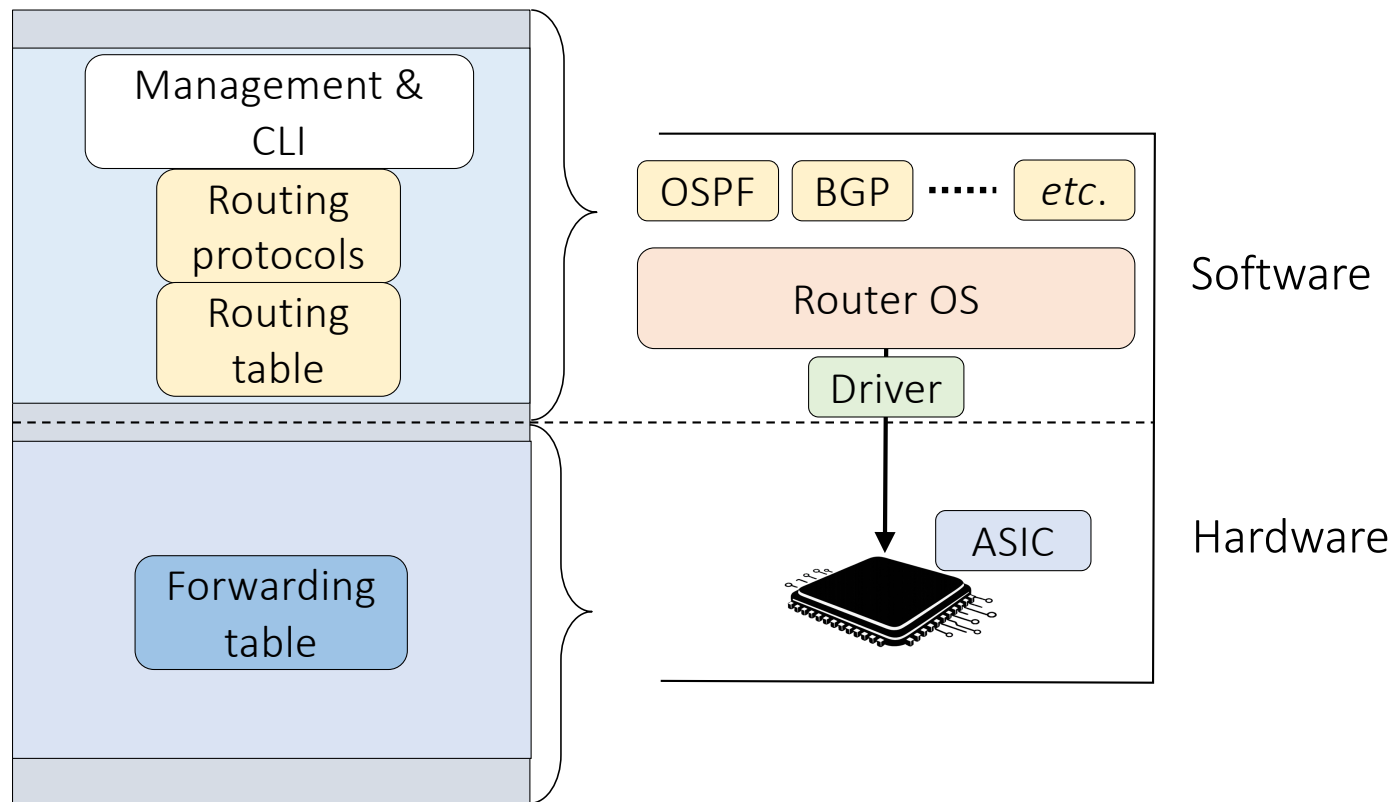


How does it look like internally?

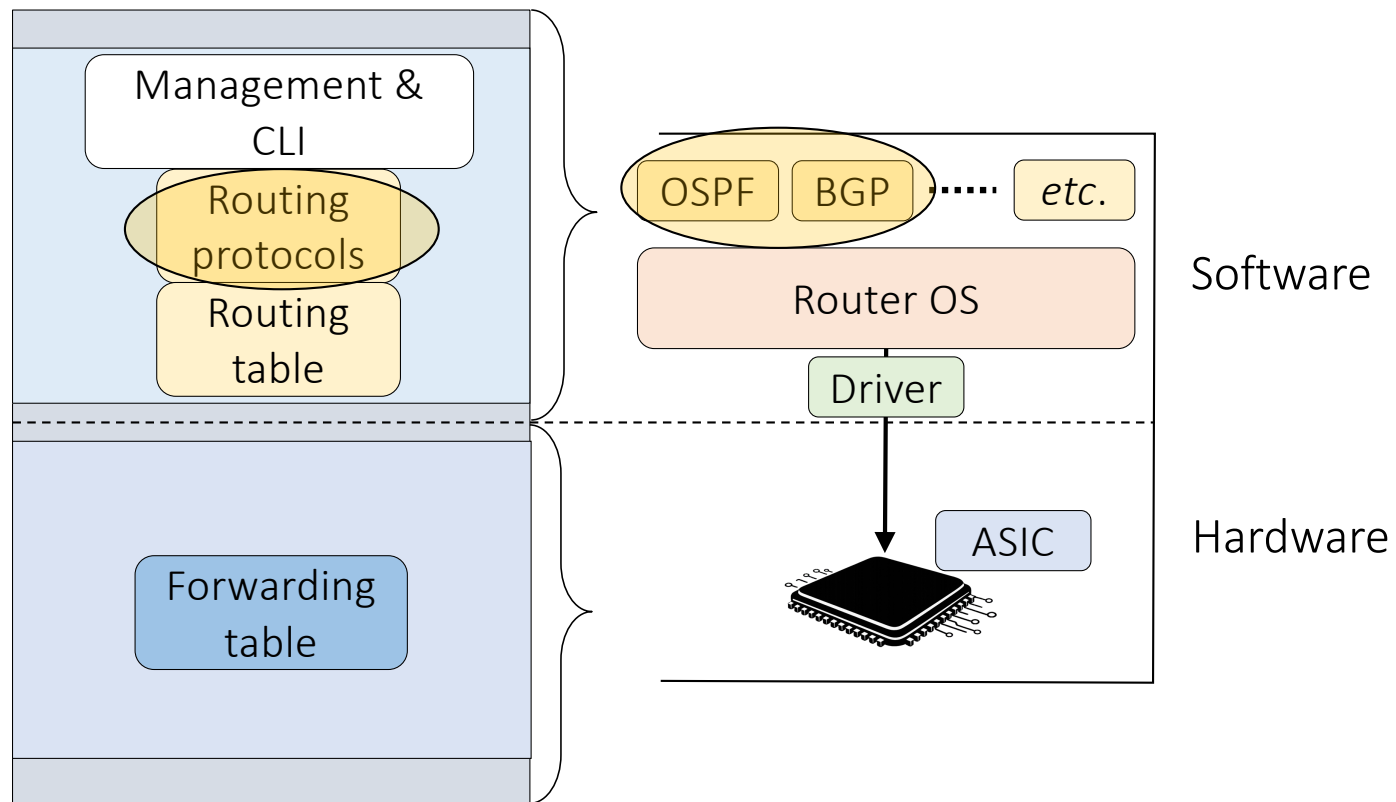
Legacy router/switch architecture



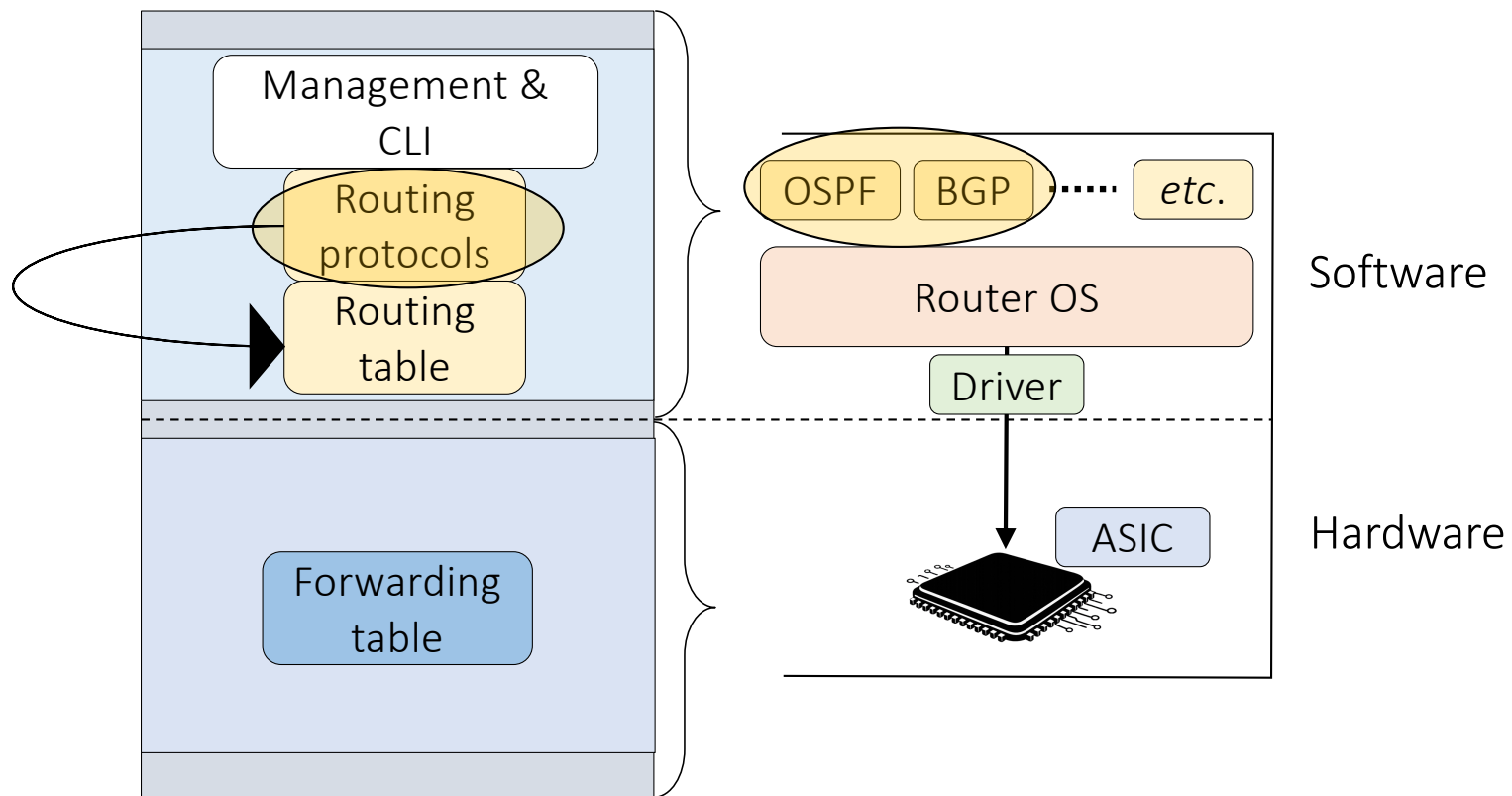
Legacy router/switch architecture



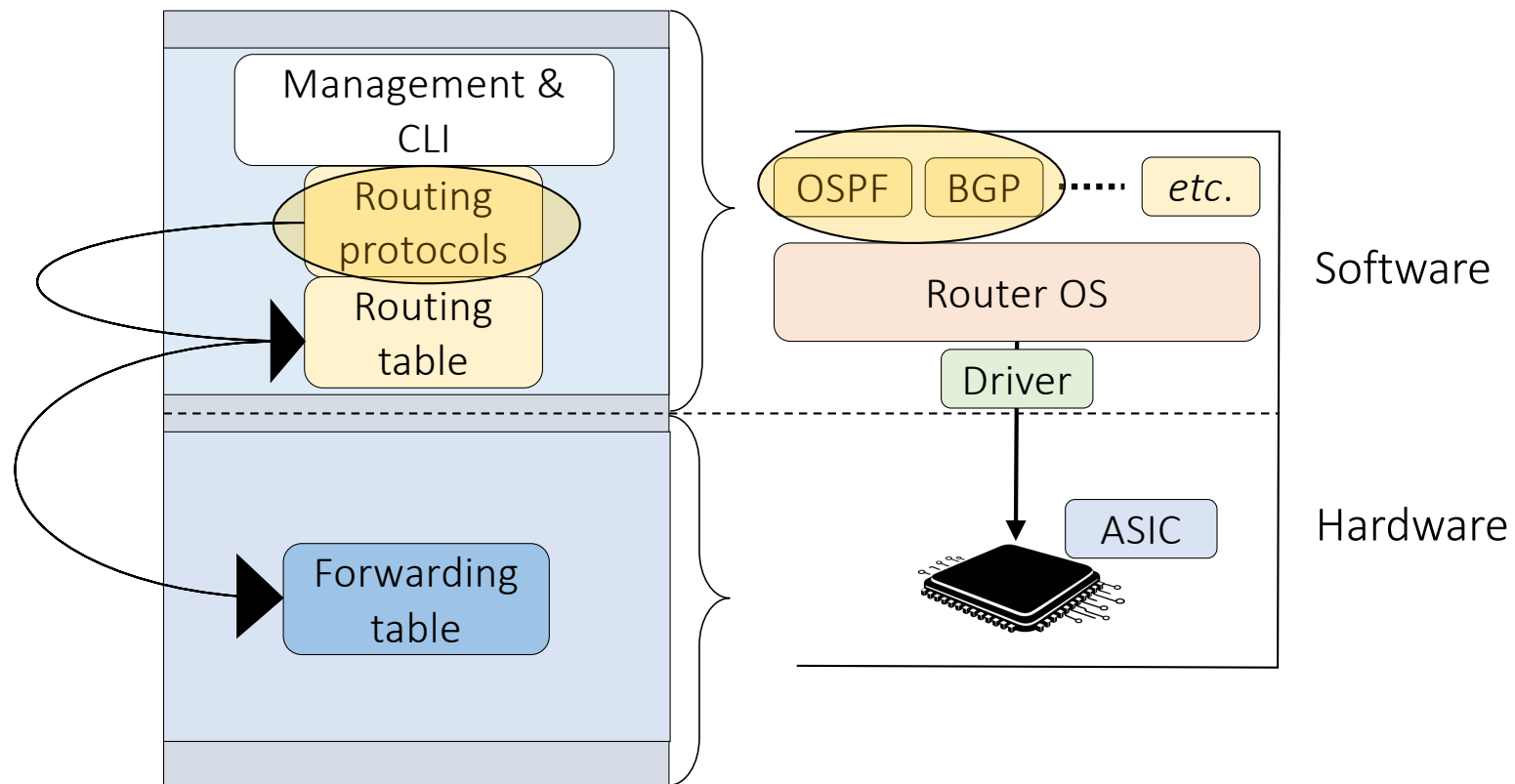
Legacy router/switch architecture



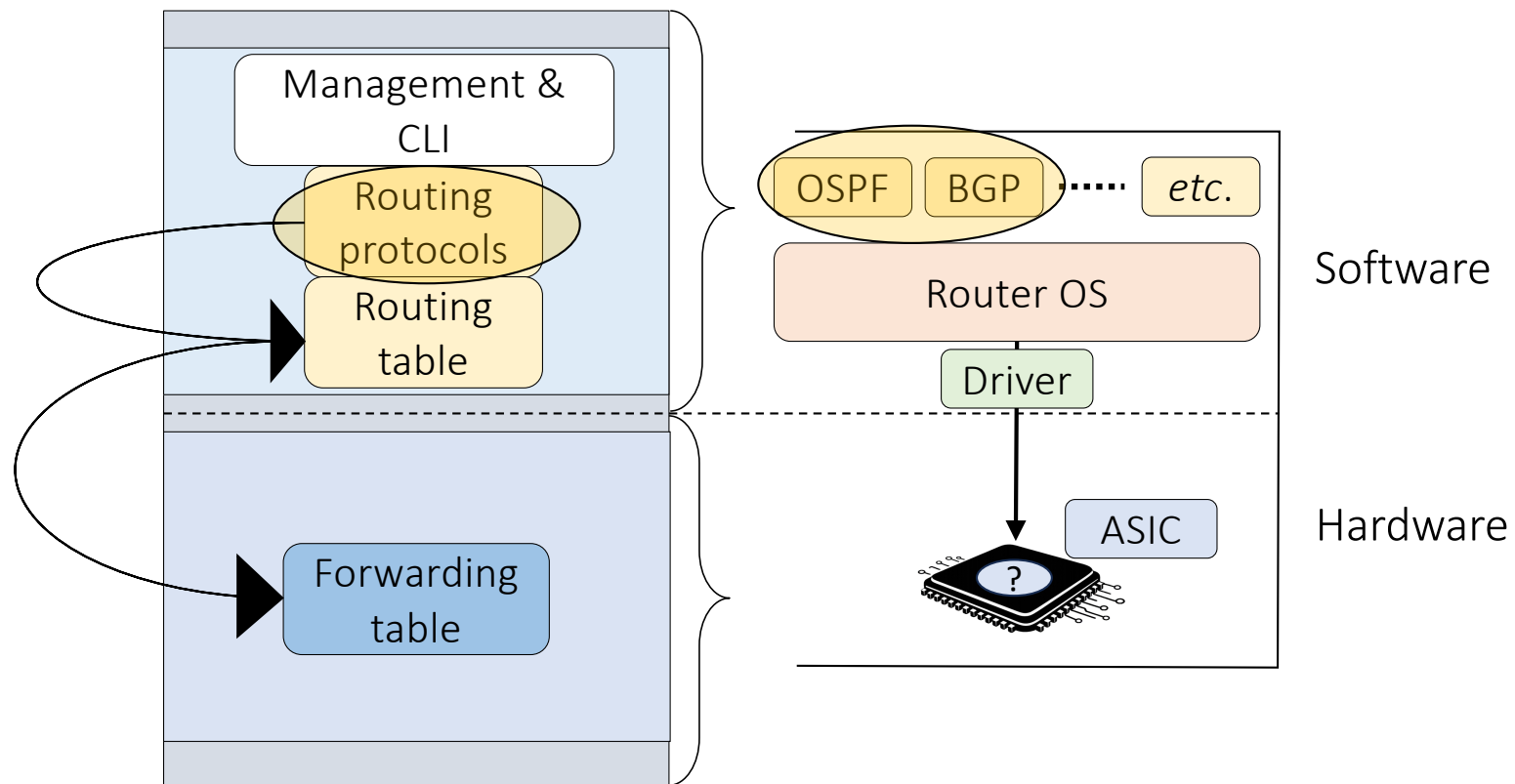
Legacy router/switch architecture



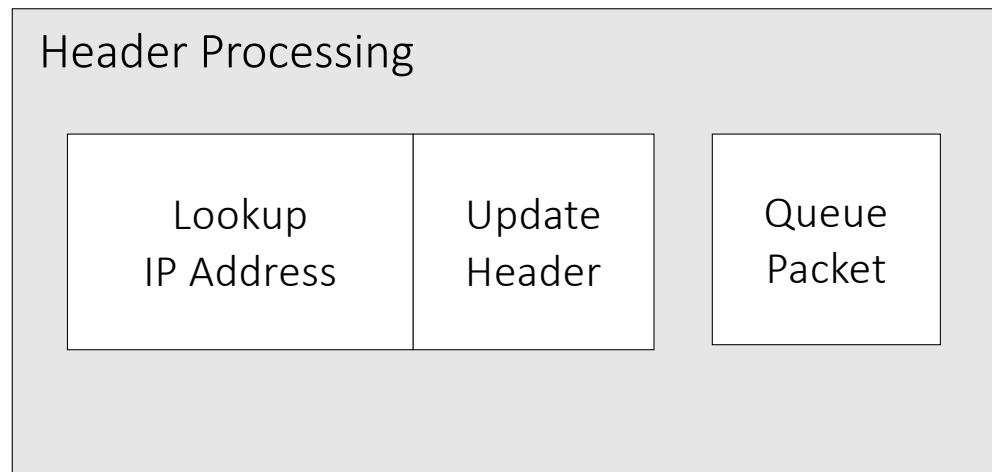
Legacy router/switch architecture



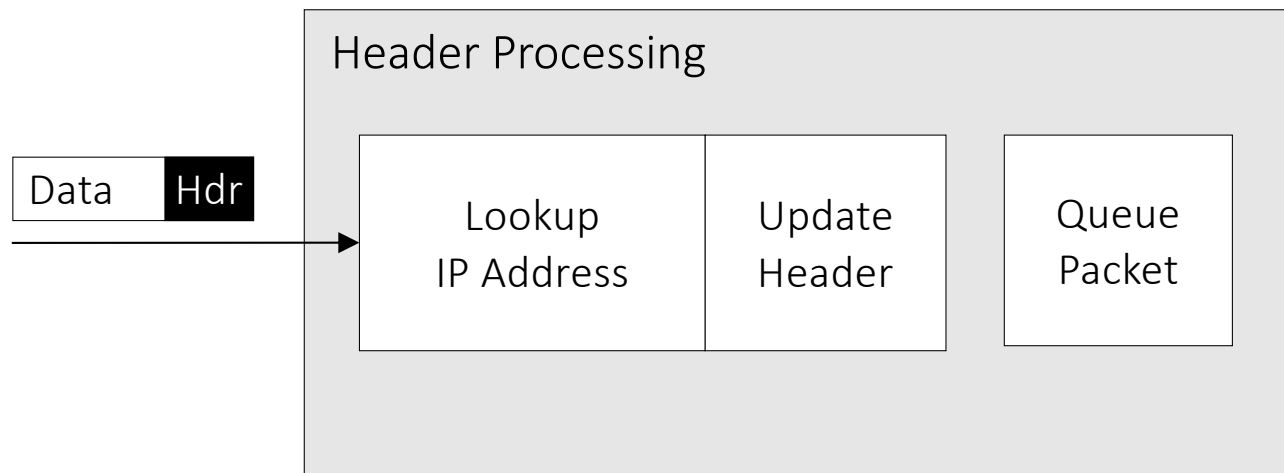
Legacy router/switch architecture



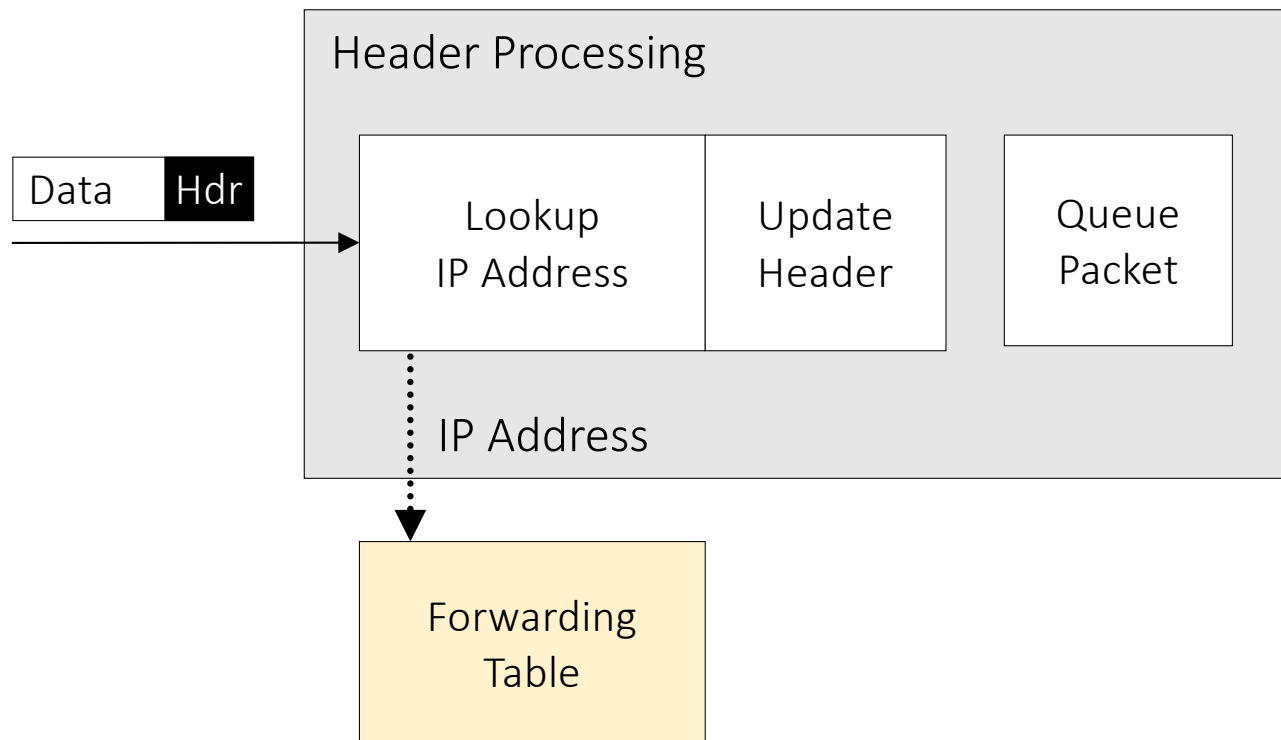
Legacy router/switch architecture



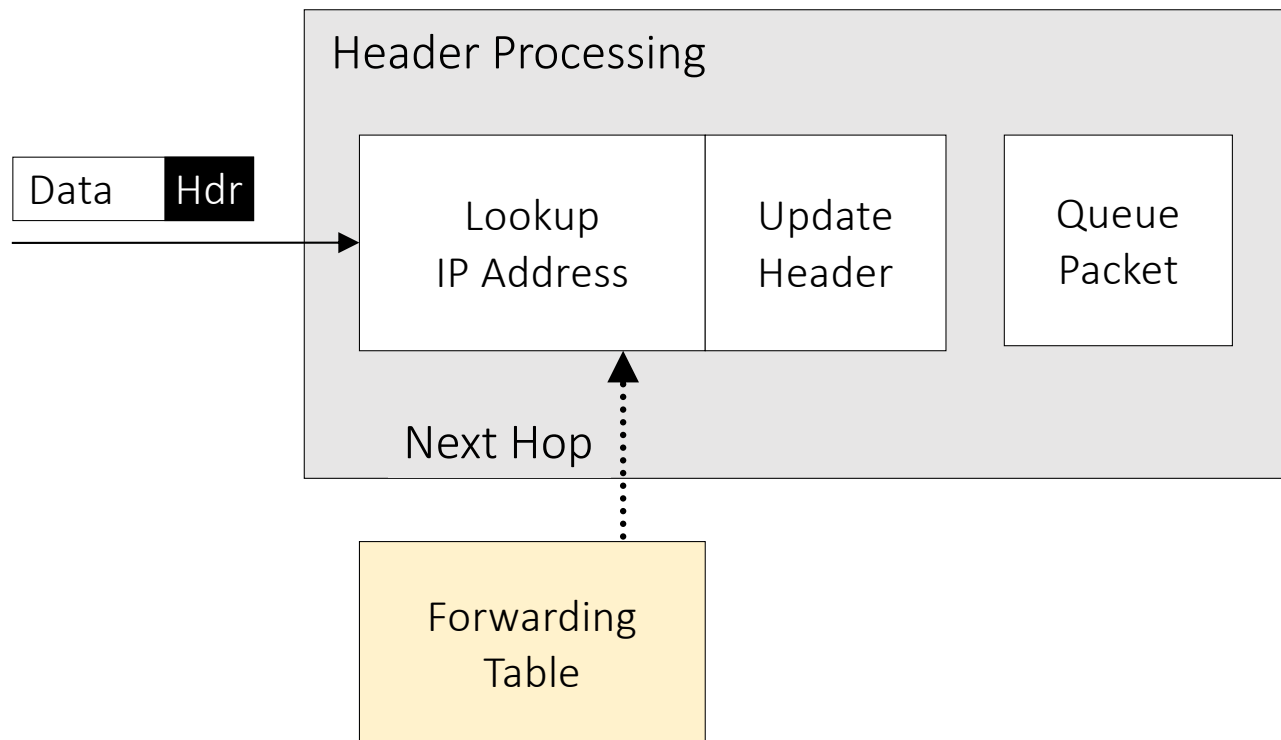
Legacy router/switch architecture



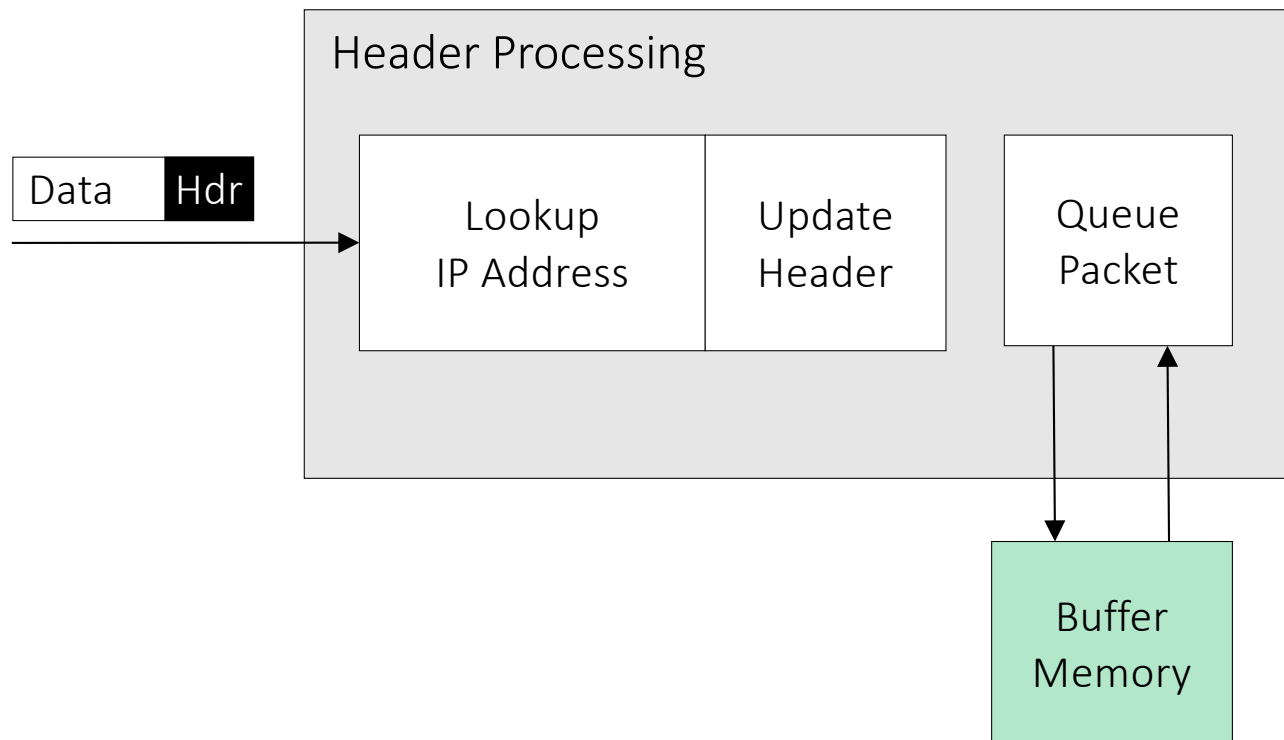
Legacy router/switch architecture



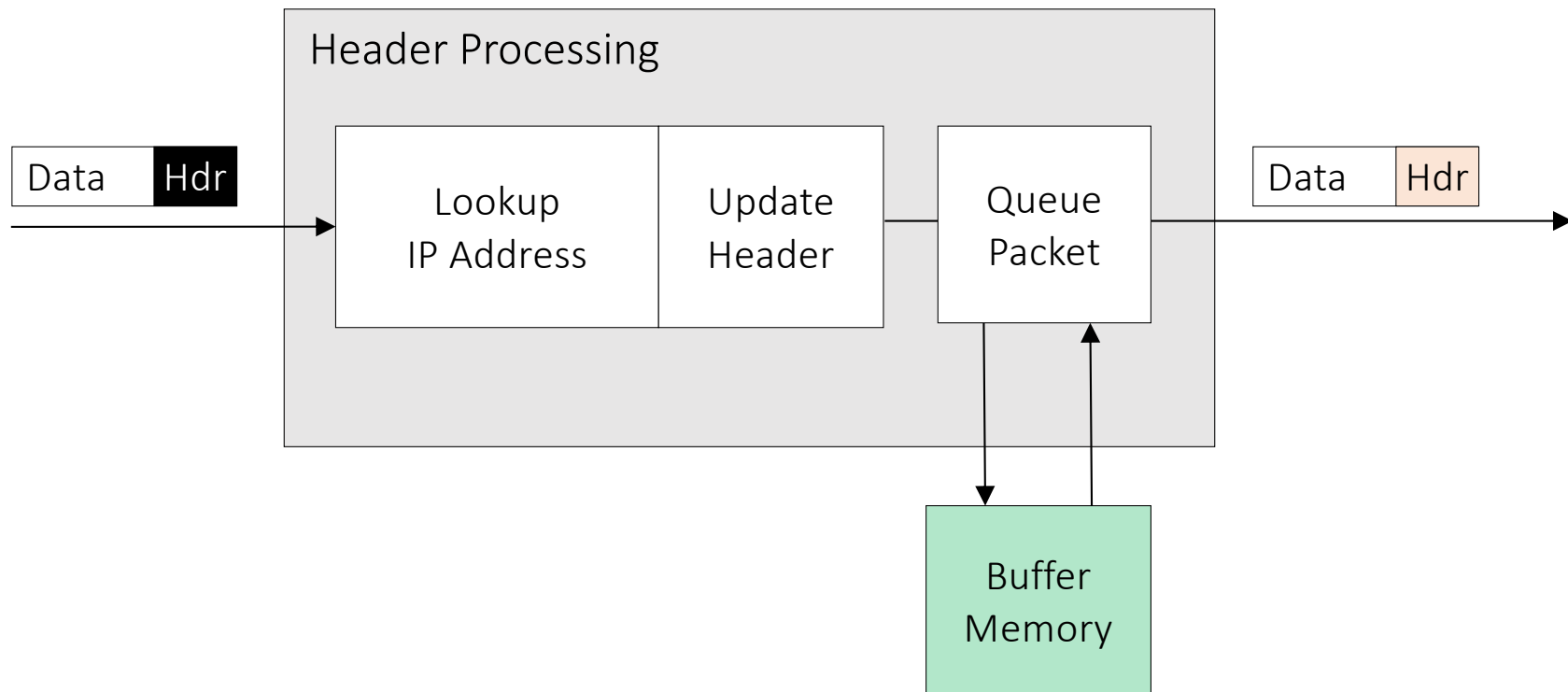
Legacy router/switch architecture



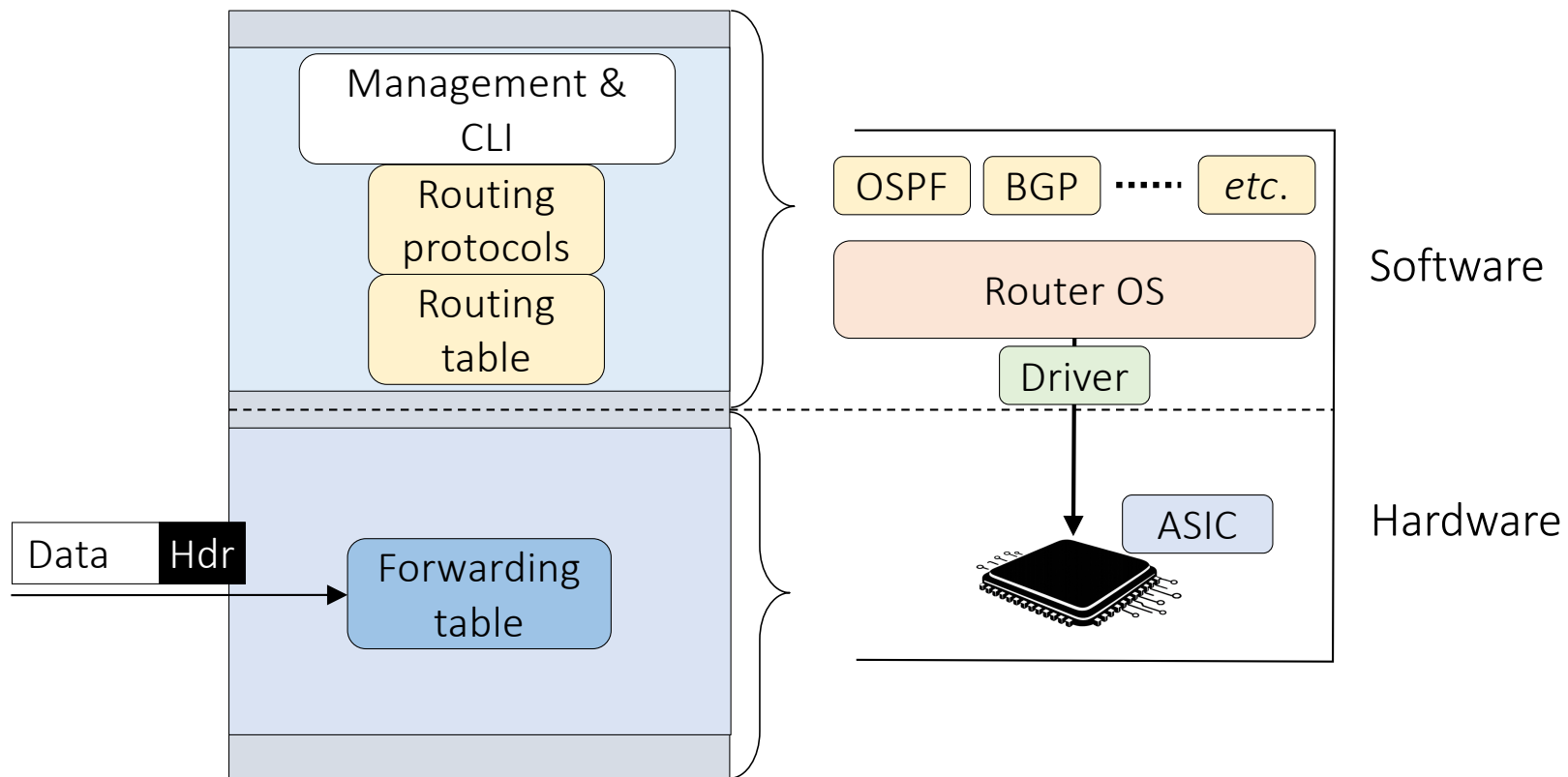
Legacy router/switch architecture



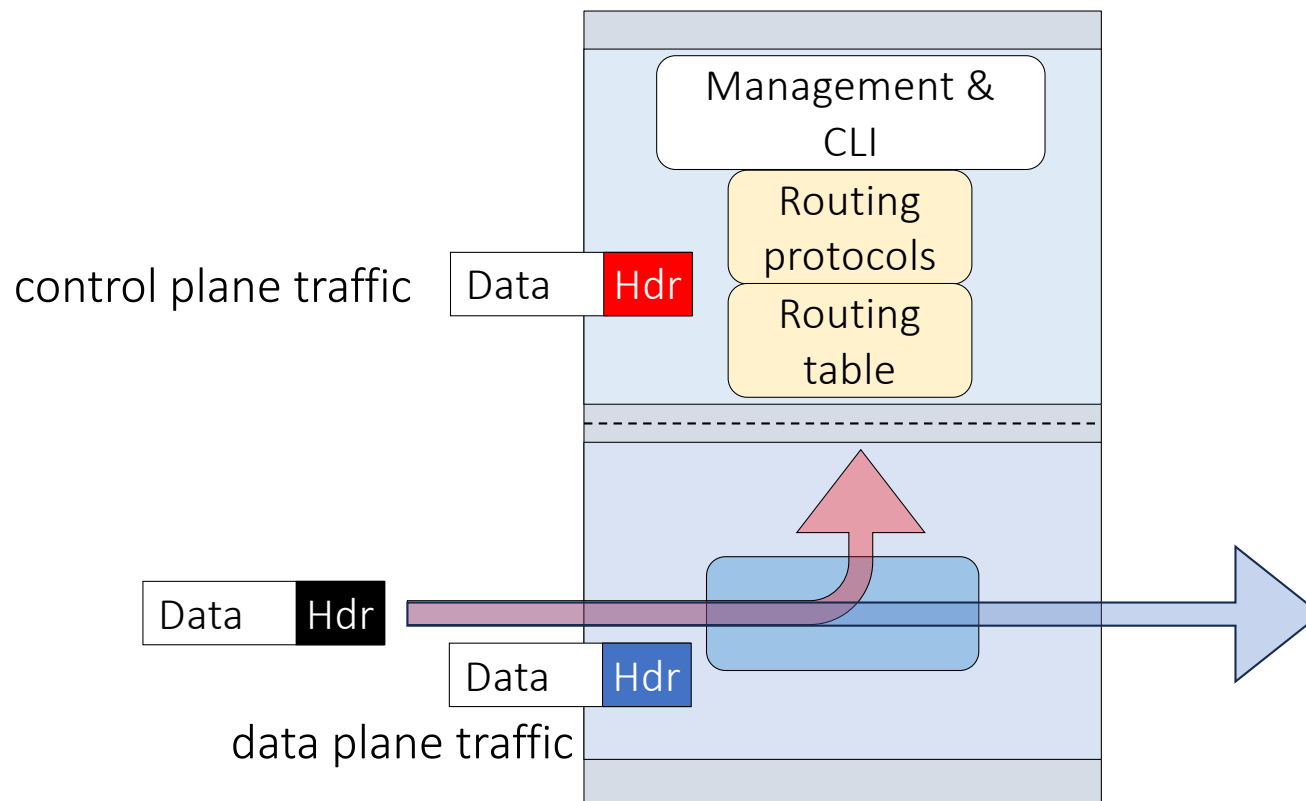
Legacy router/switch architecture



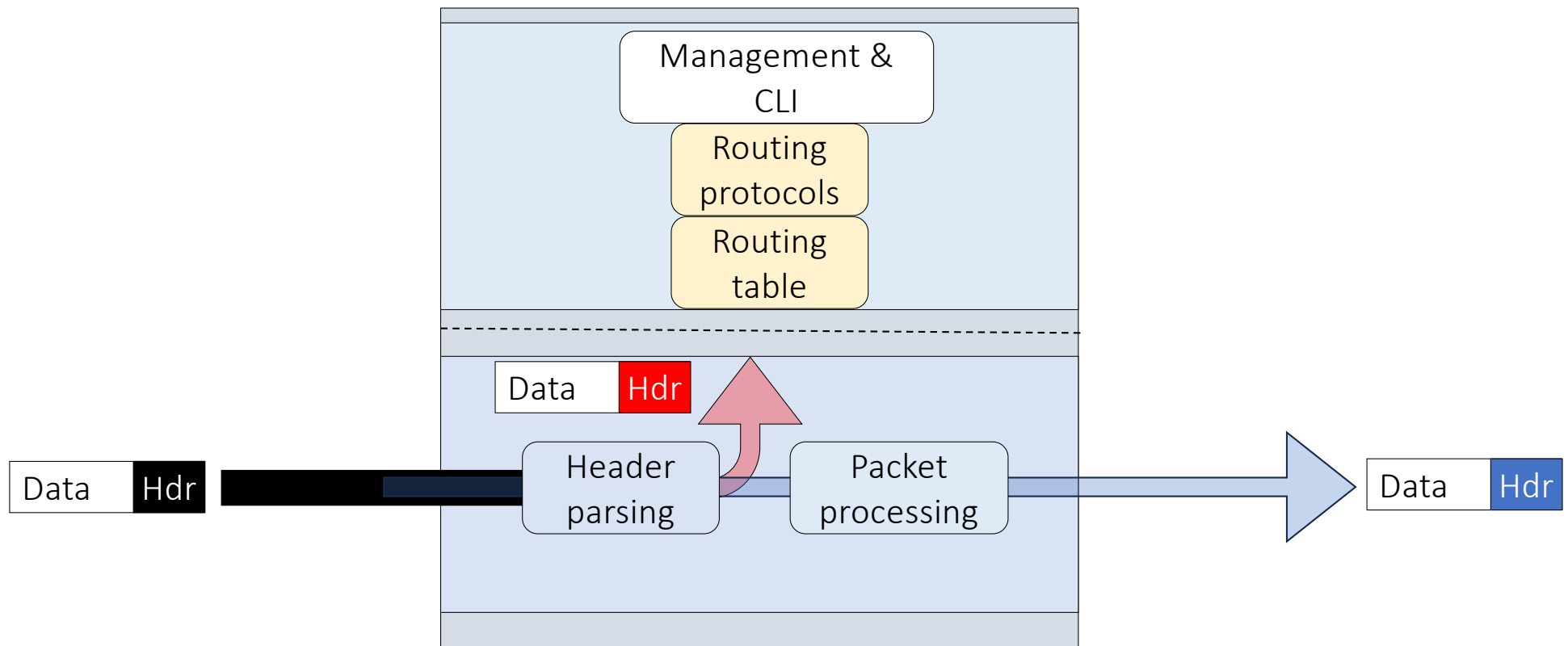
Legacy router/switch architecture



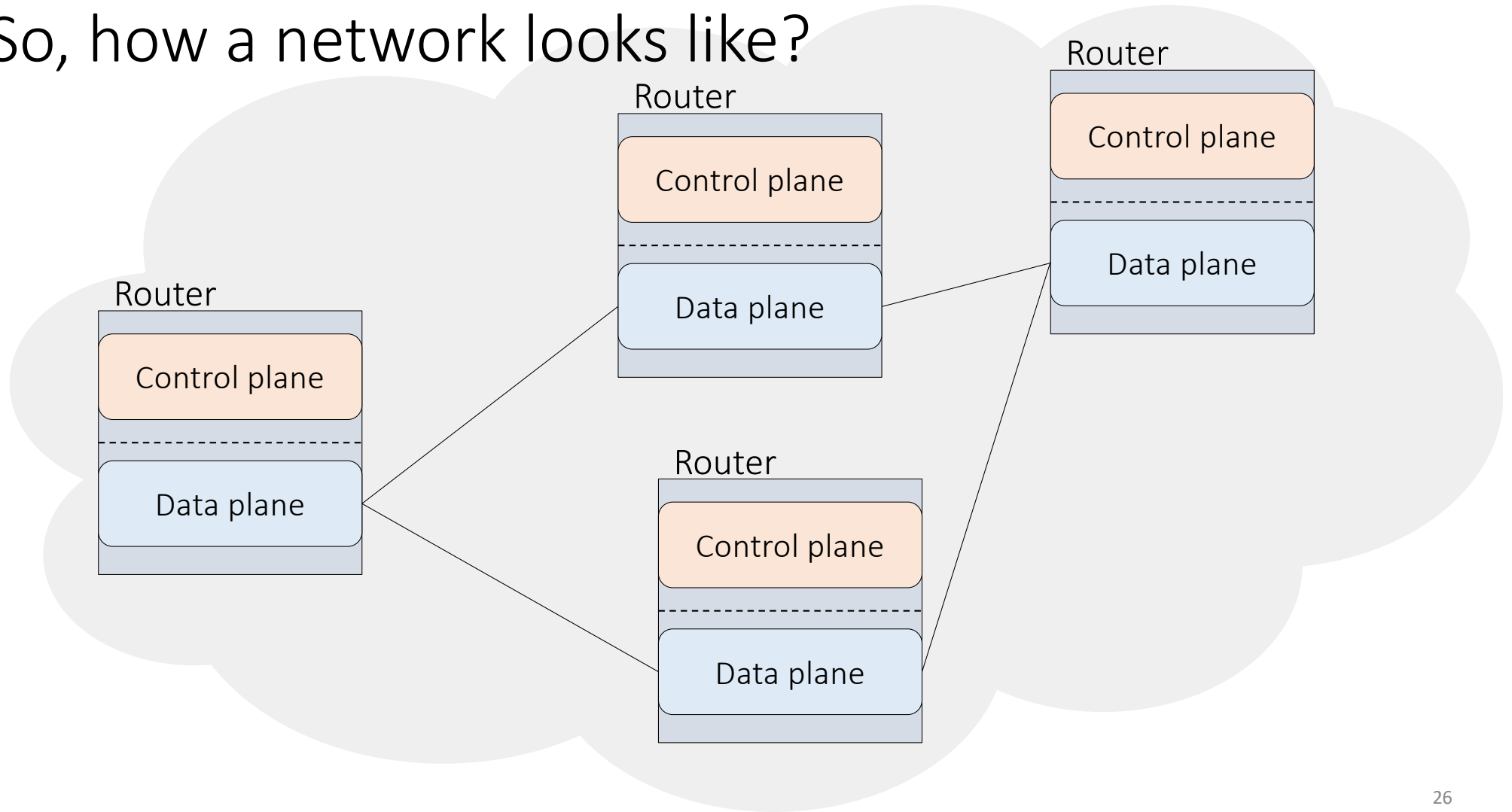
Legacy router/switch architecture



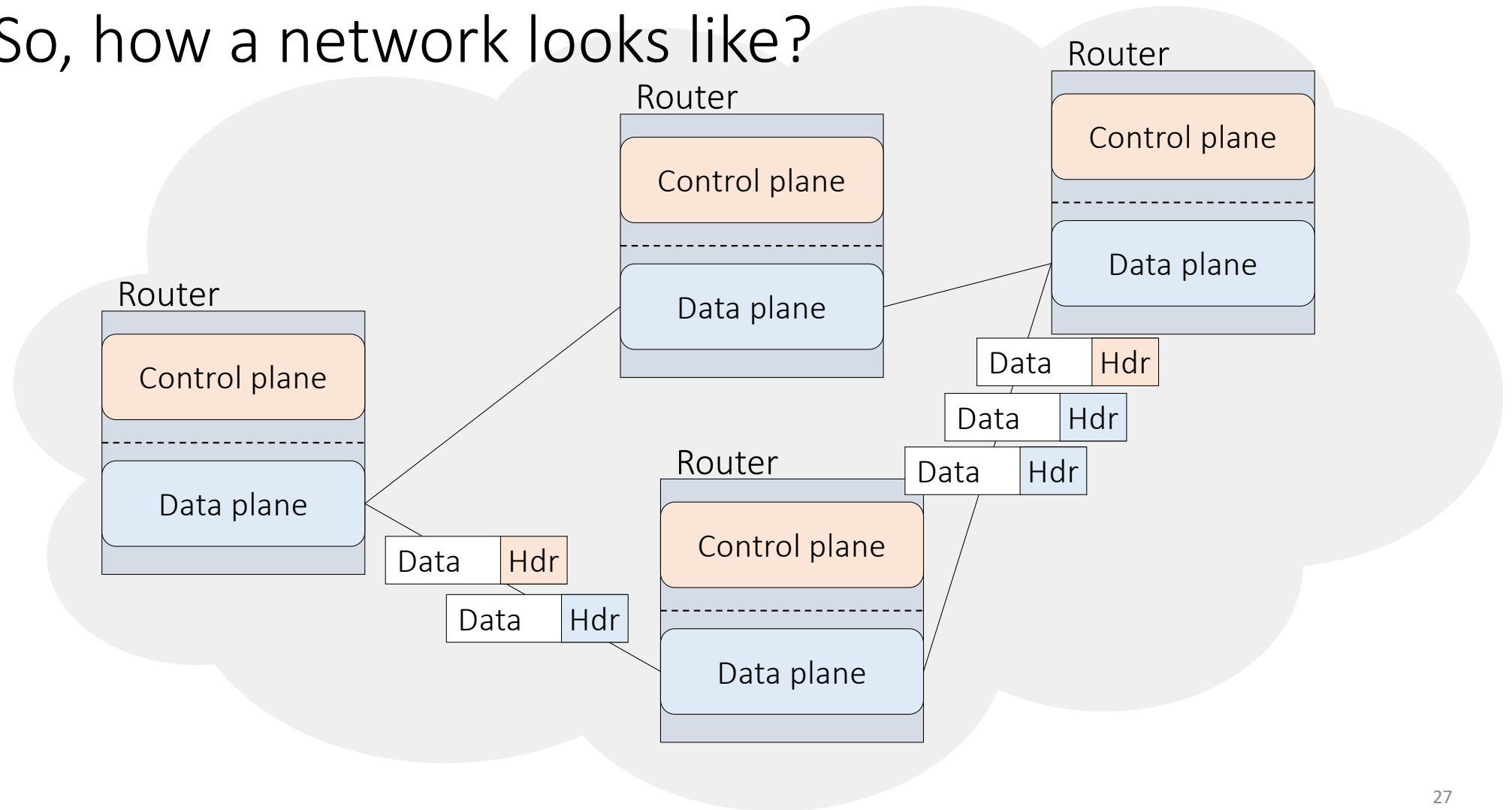
Legacy router/switch architecture



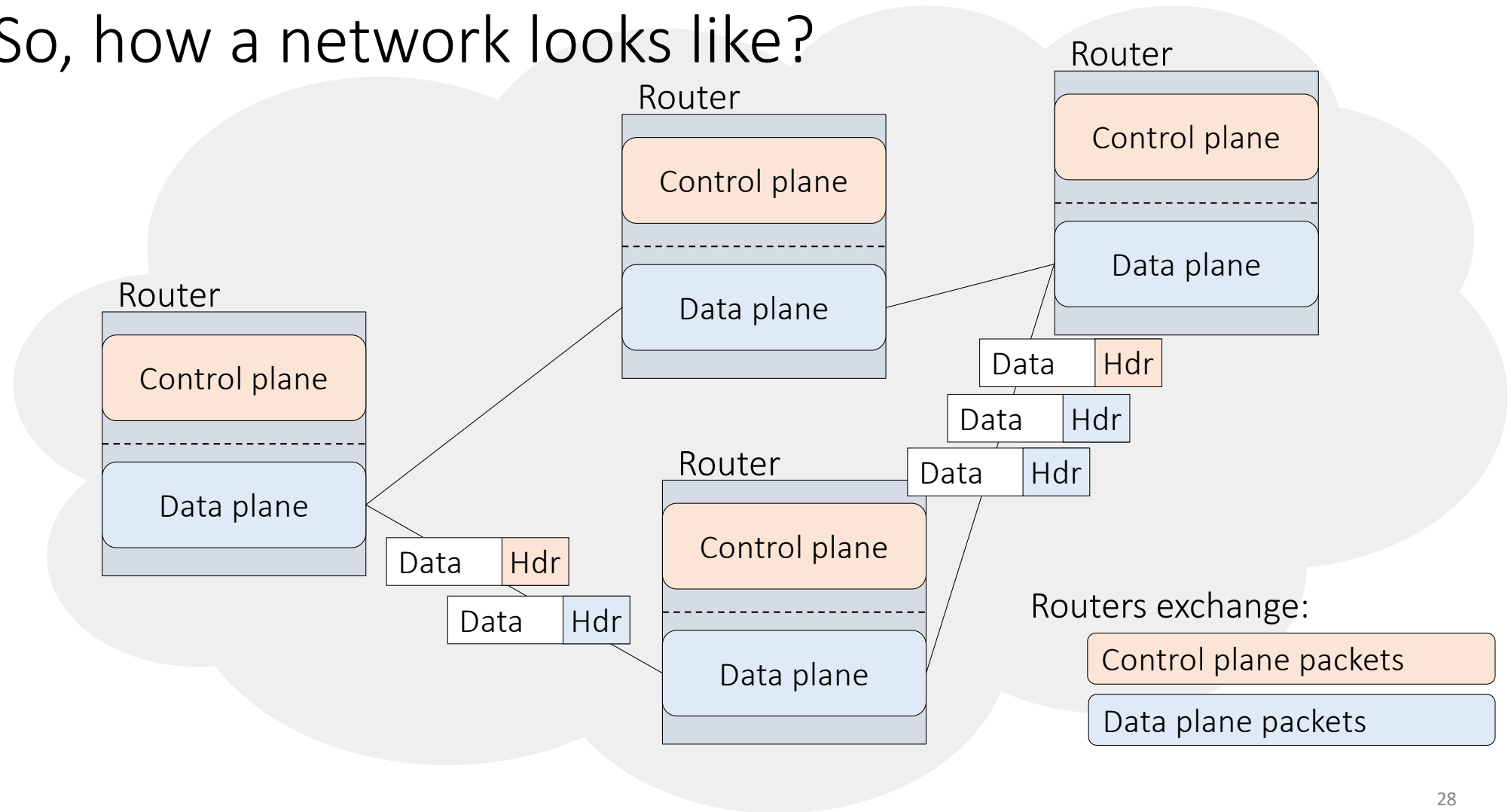
So, how a network looks like?



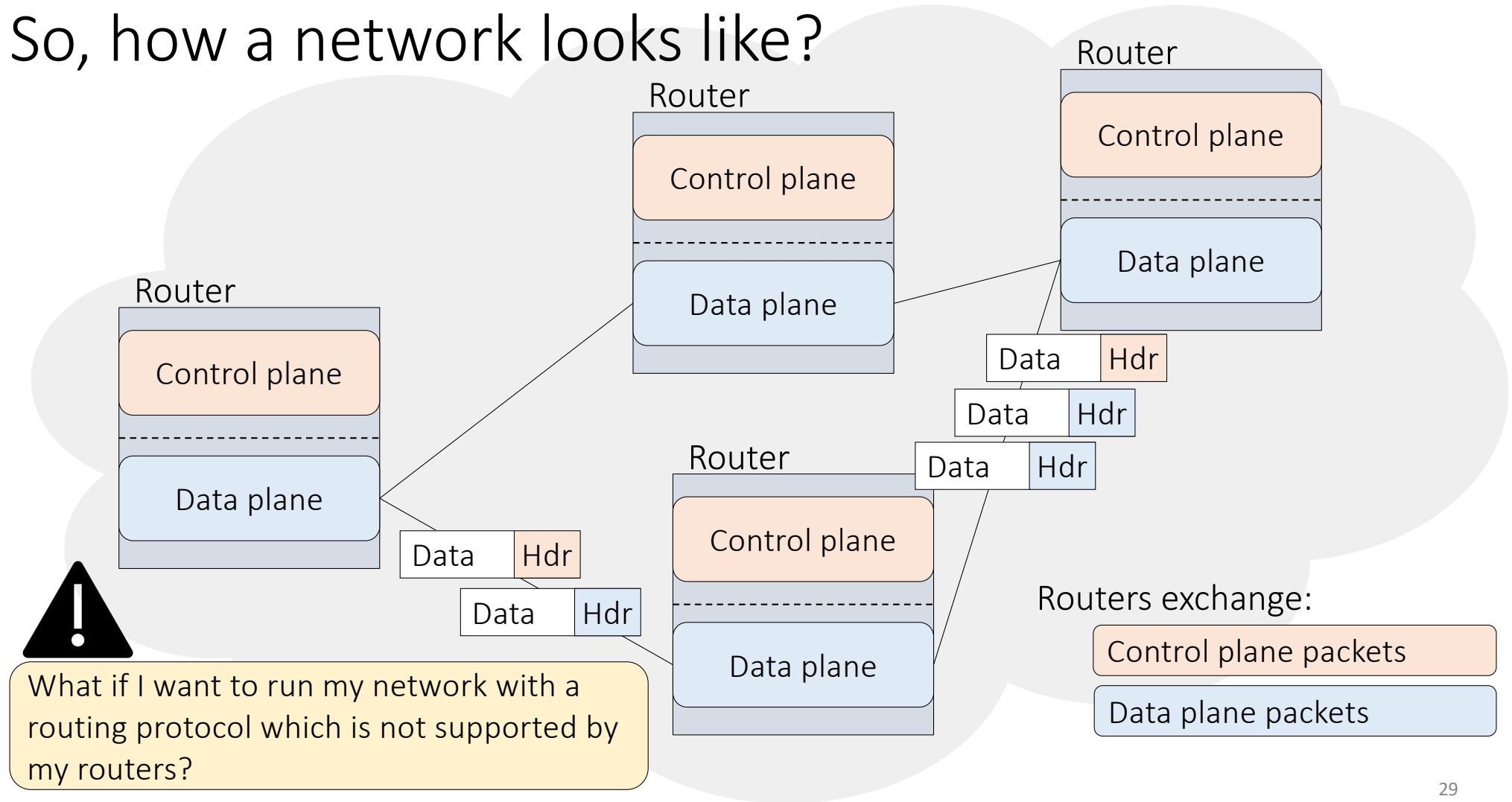
So, how a network looks like?



So, how a network looks like?

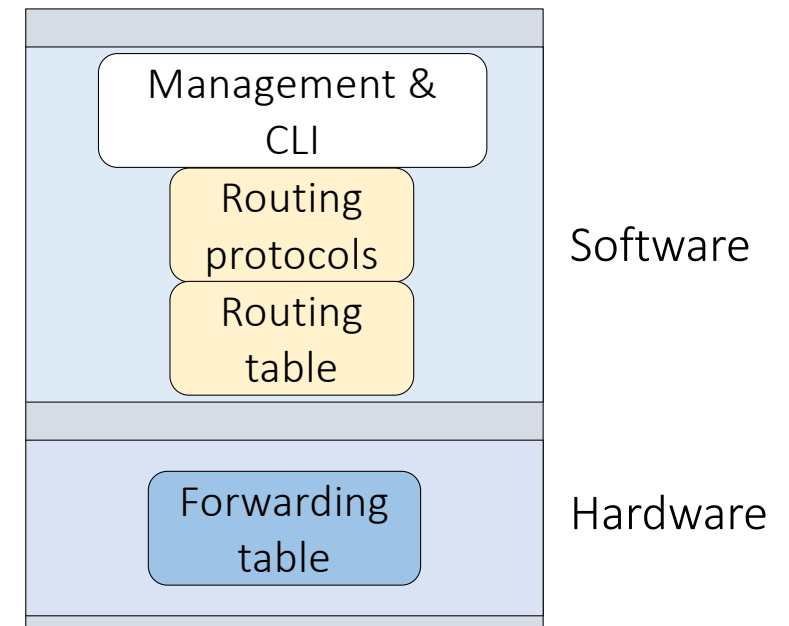


So, how a network looks like?

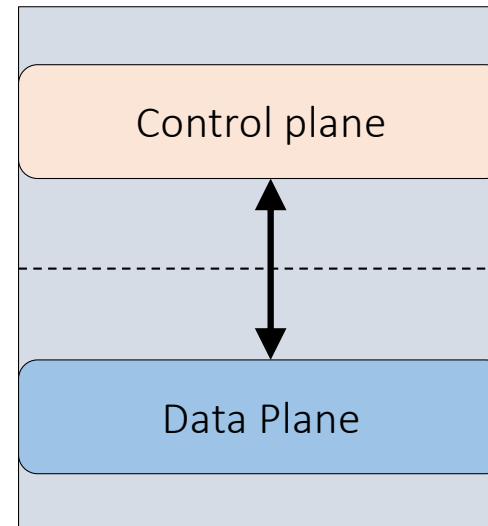


New features in the control plane

- You can potentially request a software update to your router vendor
 - Control planes are usually implemented in software
 - This can take weeks
- A software update might not be enough
 - Why?

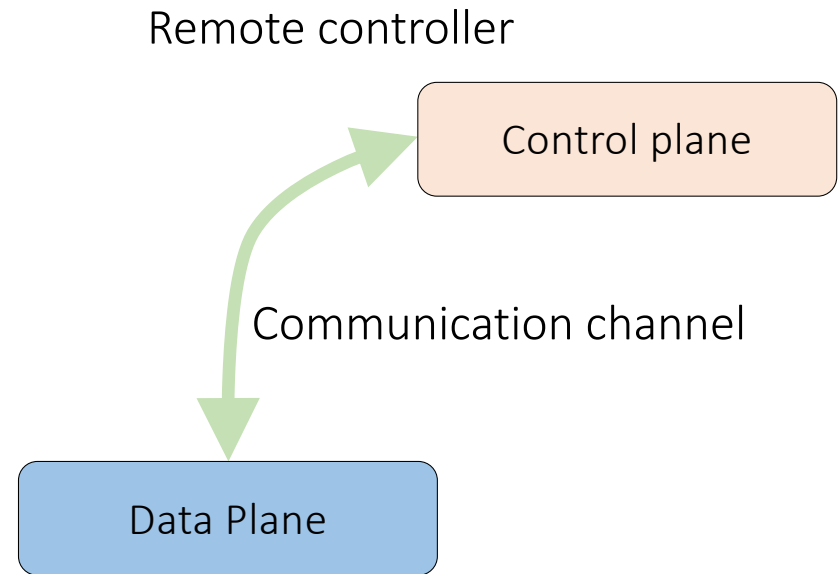


Software-Defined Networking (SDN)



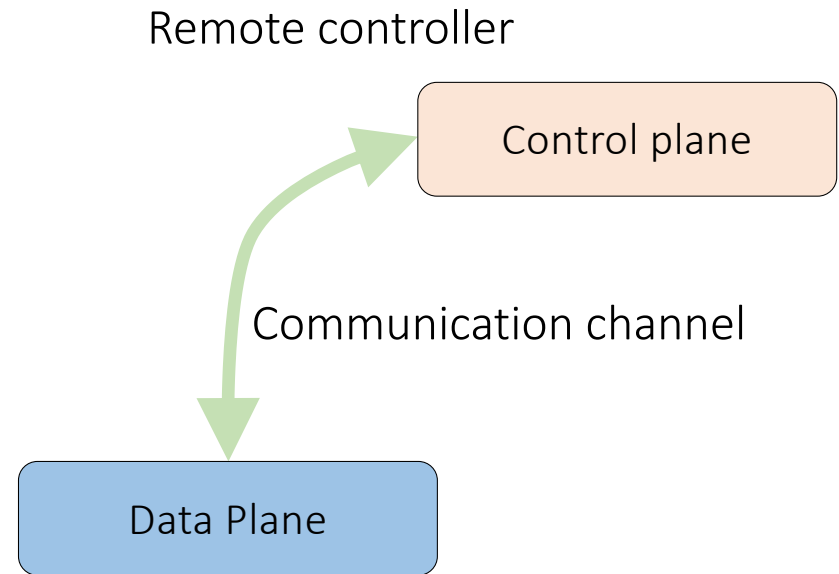
Software-Defined Networking (SDN)

- The basic idea is to decouple control plane from the data plane
- The control plane is now a remote controller
- The data plane is connected to the control plane through a communication channel



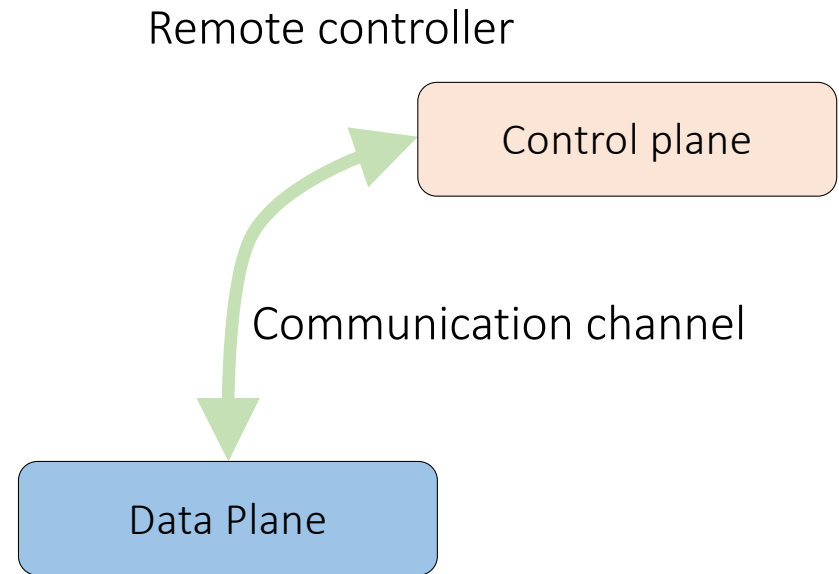
Software-Defined Networking (SDN)

- Let's give a simple interface to the data plane
- We do not need to change data plane functionality too often
- Implement the control plane in software so we can change as often as required

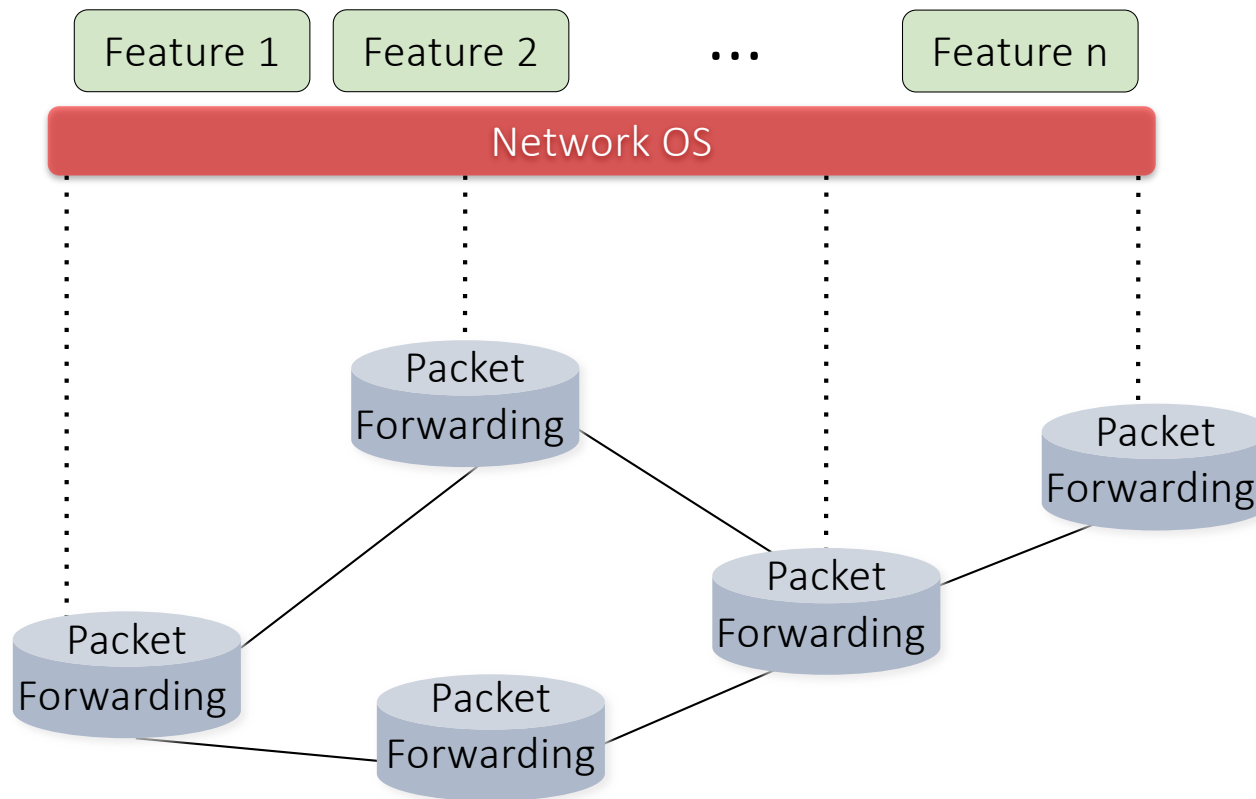


Software-Defined Networking (SDN)

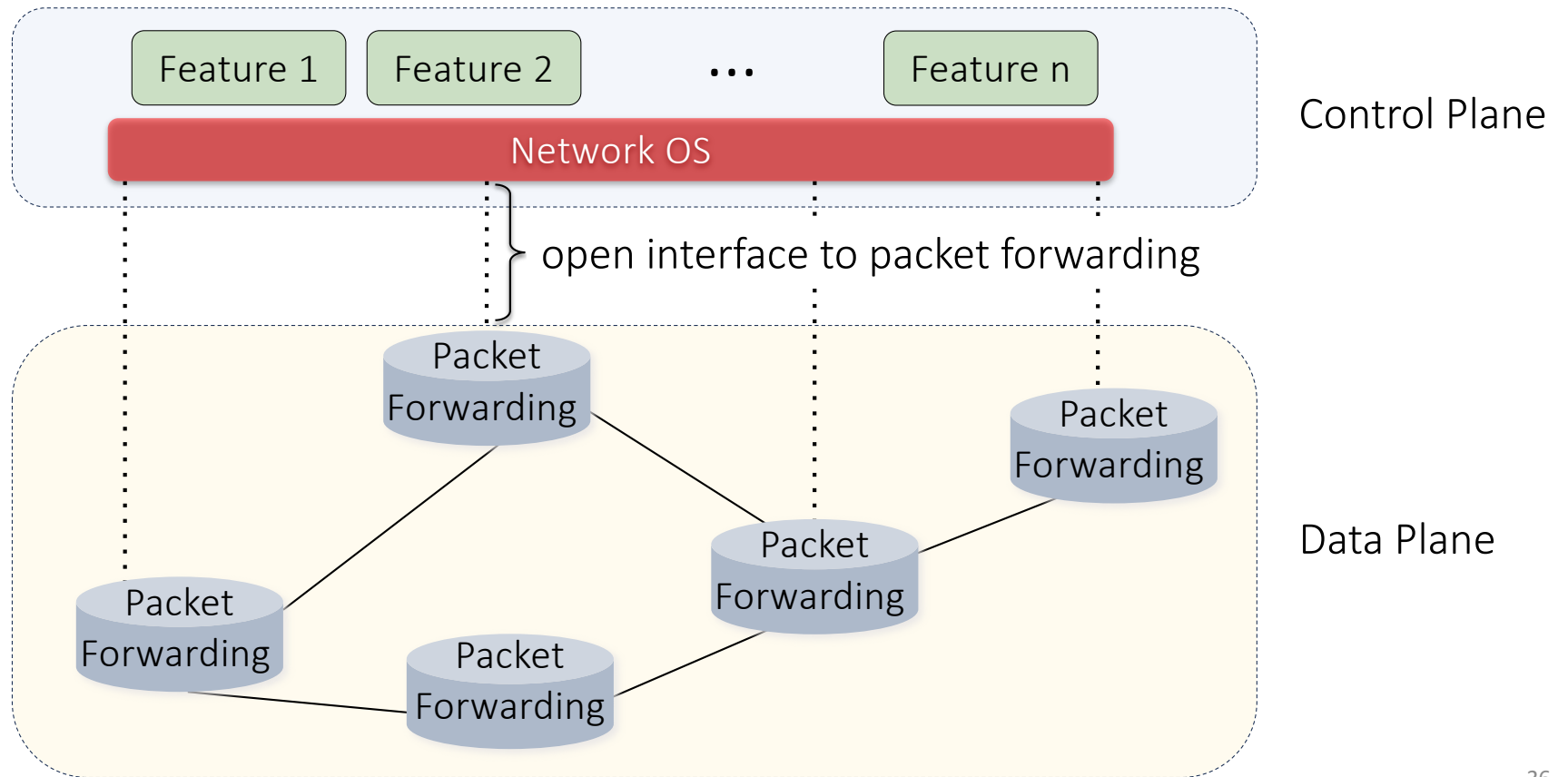
- Beyond today's monolithic network equipment
- The communication channel exposes network equipment capabilities
- It is like being able to control remotely your forwarding engine and how it treats incoming packets



Software-Defined Network

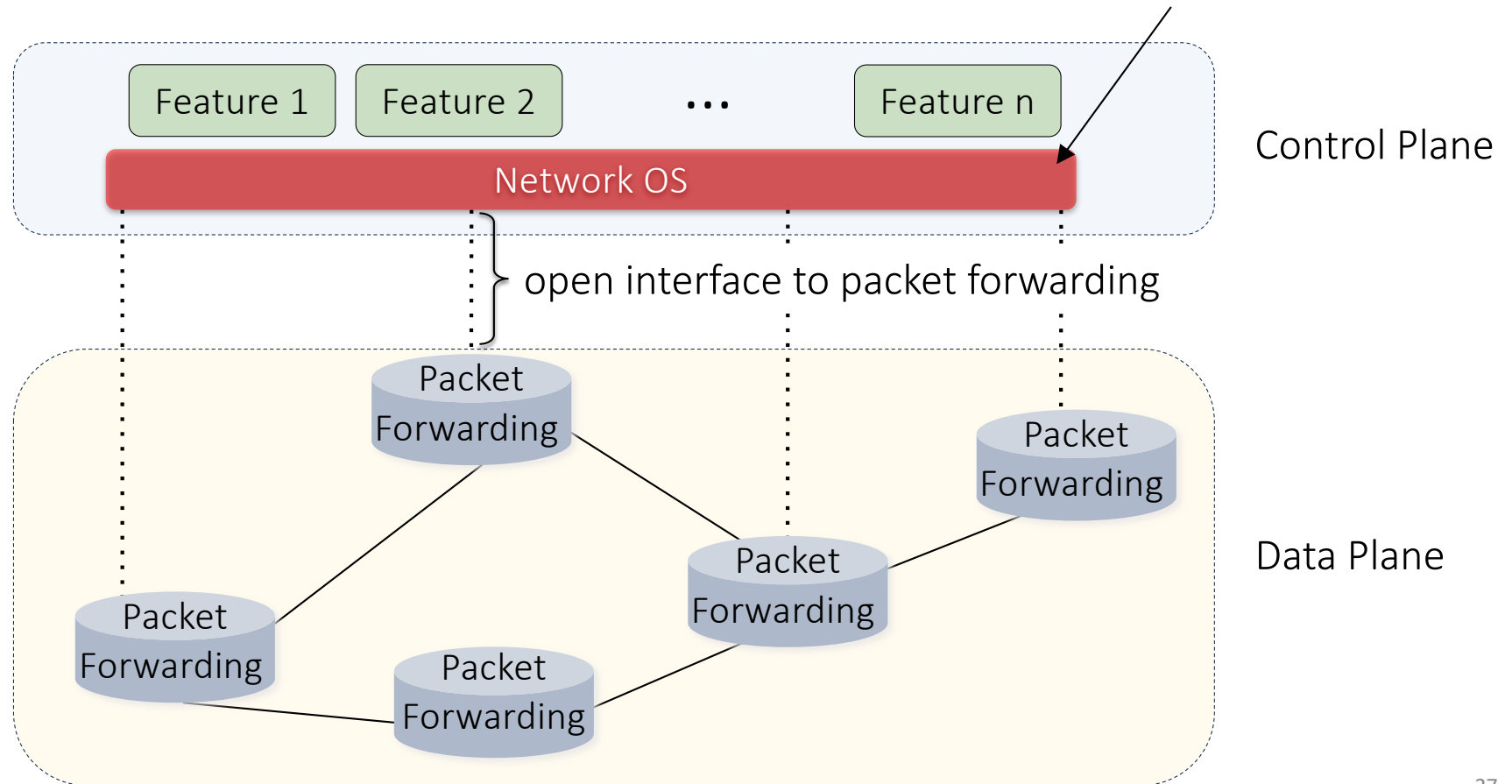


Software-Defined Network



Software-Defined Network

A Network Operating System that provides the abstractions for the features

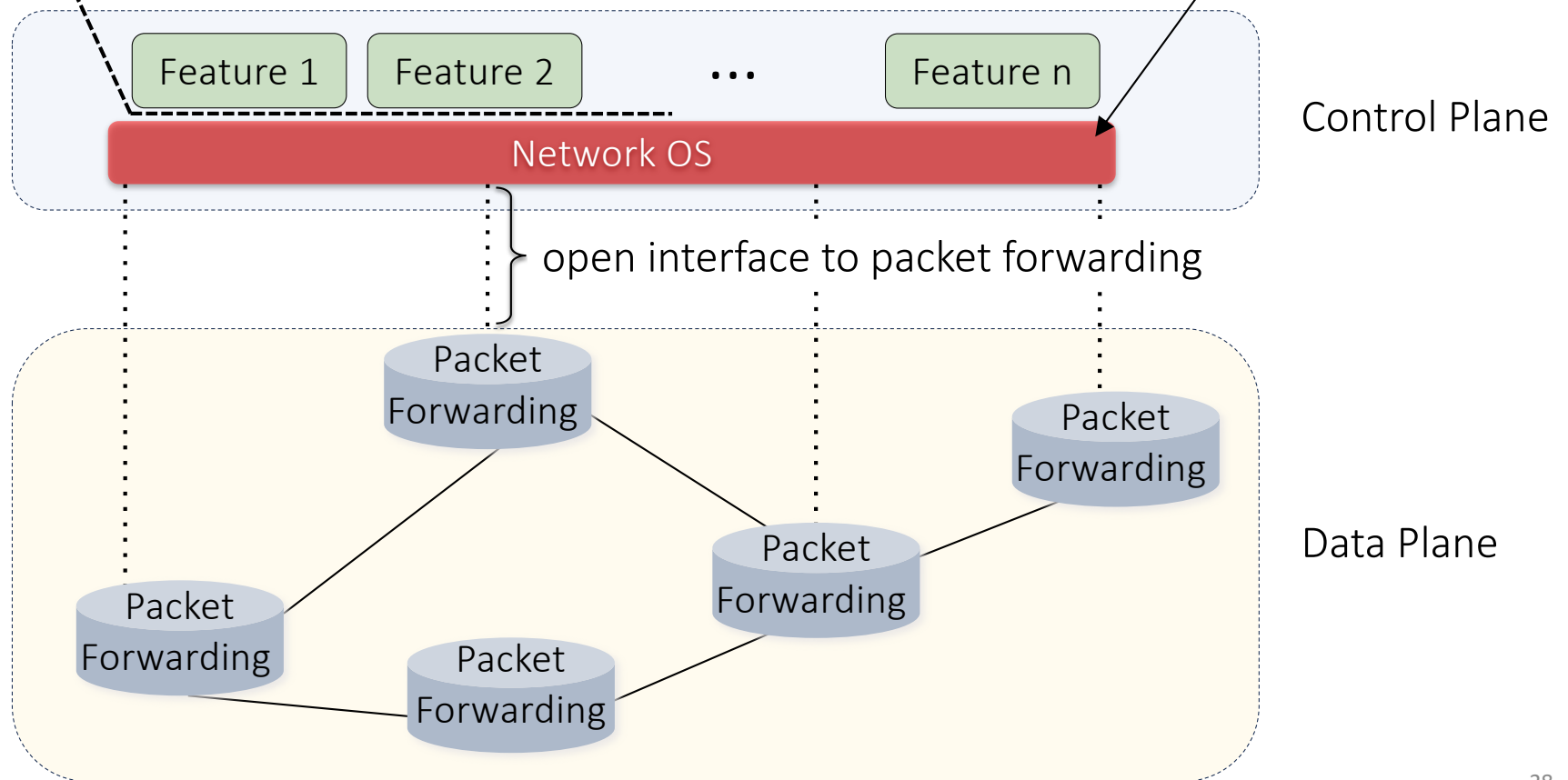


Software-Defined Network

A Network Operating System that provides the abstractions for the

Consistent, up-to-date global network view

features

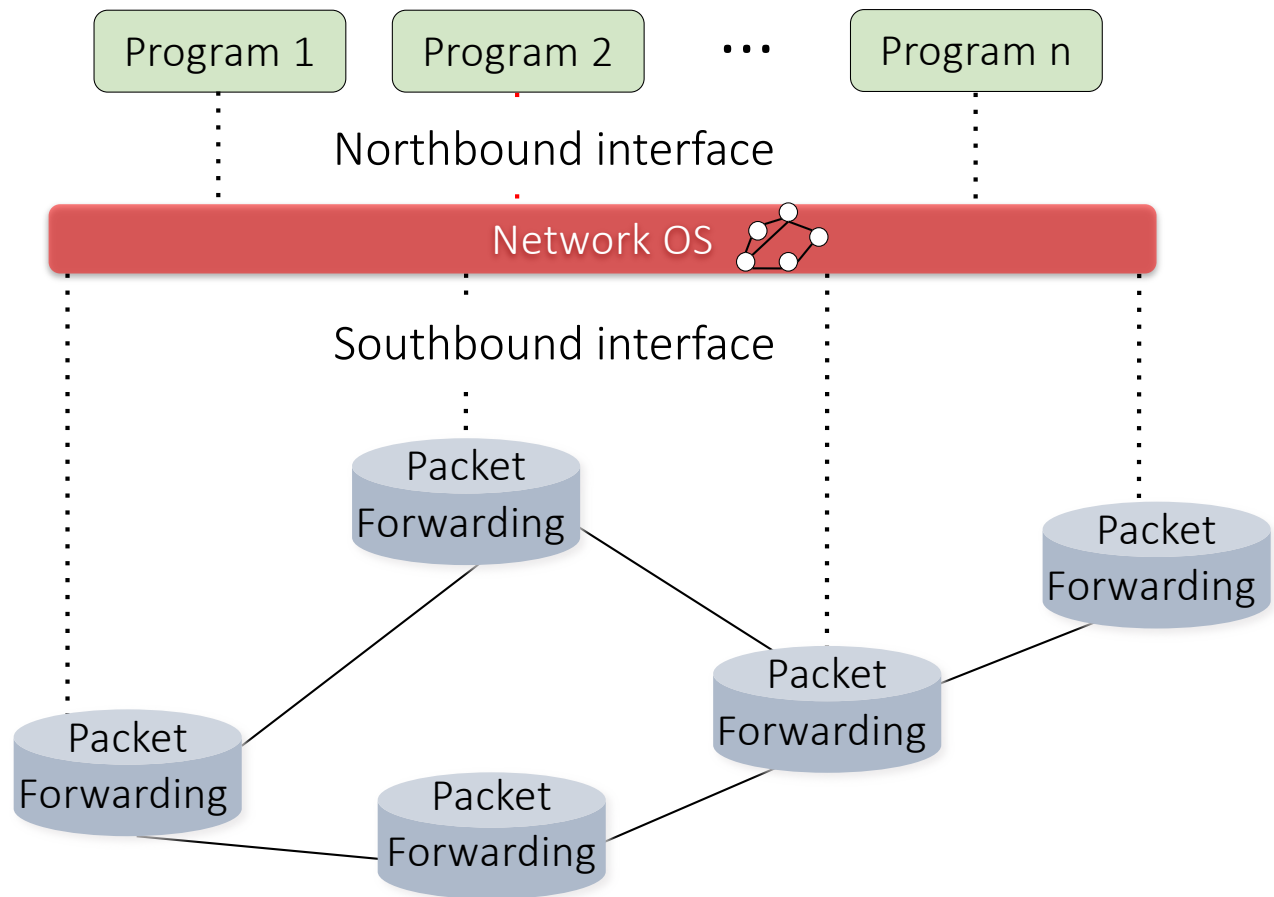


Network Operating System

- Network OS is a software that creates a consistent, up-to-date view of the network
 - It runs on servers (controllers) in the network
 - NOX (C language), Ryu (Python language), Floodlight (Java language), ONOS (Java language)
- Network OS uses forwarding abstraction to:
 - Give state information from forwarding elements
 - Give control directives to forwarding elements

Software-Defined Network

- A program or control program:
 - **Input:** global network view from the OS
 - **Output:** configuration of each of network devices
- This is done through well defined APIs



A bit of SDN history

- Ethane project at Stanford

Ethane: Taking Control of the Enterprise

Martin Casado, Michael J. Freedman,
Justin Pettit, Jianying Luo,
and Nick McKeown
Stanford University

Scott Shenker
U.C. Berkeley and ICSI

Remote controller

Control plane

Communication channel

Data Plane

ABSTRACT

This paper presents Ethane, a new network architecture for the enterprise. Ethane allows managers to define a single network-wide fine-grain policy, and then enforces it directly. Ethane couples extremely simple flow-based Ethernet switches with a centralized controller that manages the admittance and routing of flows. While radical, this design is backwards-compatible with existing hosts and switches.

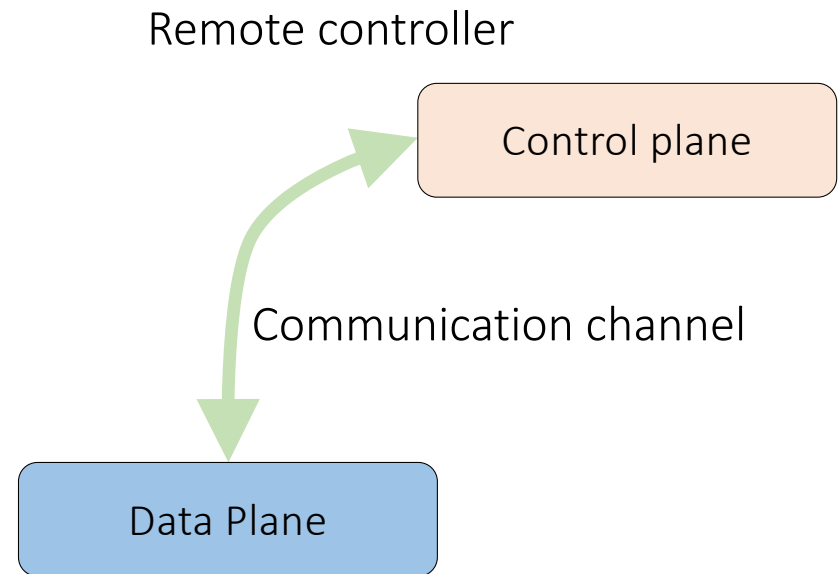
We have implemented Ethane in both hardware and software, supporting both wired and wireless hosts. Our operational Ethane network has supported over 300 hosts for the past four months in in Stanford University's network, and this deployment experience has significantly affected Ethane's design.

downtime in multi-vendor networks comes from human-error and that 80% of IT budgets is spent on maintenance and operations [16].

There have been many attempts to make networks more manageable and more secure. One approach introduces proprietary middleboxes that can exert their control effectively only if placed at network choke-points. If traffic accidentally flows (or is maliciously diverted) around the middlebox, the network is no longer managed nor secure [25]. Another approach is to add functionality to existing networks—to provide tools for diagnosis, to offer controls for VLANs, access-control lists, and filters to isolate users, to instrument the routing and spanning tree algorithms to support better connectivity management, and then to collect packet traces to allow auditing. This can be done by adding a new layer of protocols, scripts, and applications [1, 10] that help automate configuration

A bit of SDN history

- Ethane project at Stanford
- Later this project became OpenFlow



A bit of SDN history

- Ethane project at Stanford
- Later this project led to OpenFlow

OpenFlow: Enabling Innovation in Campus Networks

Nick McKeown
Stanford University

Tom Anderson
University of Washington

Hari Balakrishnan
MIT

Guru Parulkar
Stanford University

Larry Peterson
Princeton University

Jennifer Rexford
Princeton University

Scott Shenker
University of California,
Berkeley

Jonathan Turner
Washington University in
St. Louis

This article is an editorial note submitted to CCR. It has NOT been peer reviewed.
Authors take full responsibility for this article's technical content.
Comments can be posted through CCR Online.

ABSTRACT

This whitepaper proposes OpenFlow: a way for researchers to run experimental protocols in the networks they use every day. OpenFlow is based on an Ethernet switch, with an internal flow-table, and a standardized interface to add and remove flow entries. Our goal is to encourage networking vendors to add OpenFlow to their switch products for deployment in college campus backbones and wiring closets. We believe that OpenFlow is a pragmatic compromise: on one hand, it allows researchers to run experiments on heterogeneous switches in a uniform way at line-rate and with high port-density; while on the other hand, vendors do not need to expose the internal workings of their switches. In addition to allowing researchers to evaluate their ideas in real-world traffic settings, OpenFlow could serve as a useful campus component in proposed large-scale testbeds like GENI. Two buildings at Stanford University will soon run OpenFlow networks, using commercial Ethernet switches and routers. We will work to encourage deployment at other schools; and we encourage you to consider deploying OpenFlow in your university network too.

to experiment with production traffic, which have created an exceedingly high barrier to entry for new ideas. Today, there is almost no practical way to experiment with new network protocols (e.g., new routing protocols, or alternatives to IP) in sufficiently realistic settings (e.g., at scale carrying real traffic) to gain the confidence needed for their widespread deployment. The result is that most new ideas from the networking research community go untried and untested; hence the commonly held belief that the network infrastructure has "ossified".

Having recognized the problem, the networking community is hard at work developing programmable networks, such as GENI [1] a proposed nationwide research facility for experimenting with new network architectures and distributed systems. These programmable networks call for programmable switches and routers that (using *virtualization*) can process packets for multiple isolated experimental networks simultaneously. For example, in GENI it is envisaged that a researcher will be allocated a *slice* of resources across the whole network, consisting of a portion of network links, packet processing elements (e.g. routers) and end-hosts; researchers program their slices to behave as

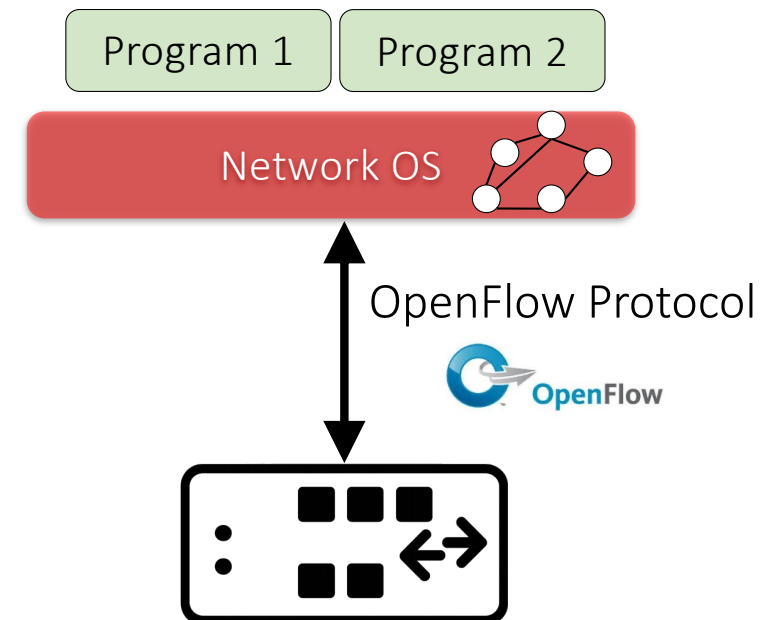
er

Control plane

data channel

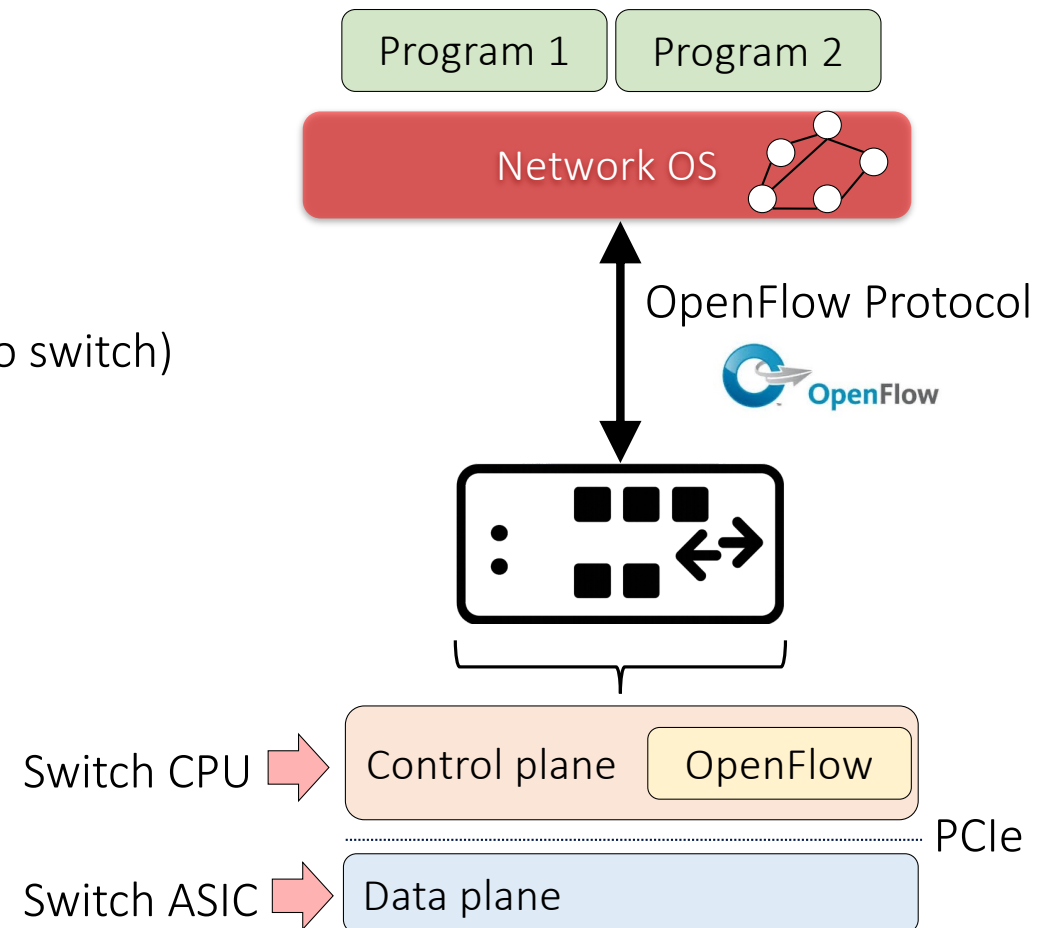
OpenFlow

- OpenFlow is a realisation of SDN
- It is a set of protocols and APIs (controller to switch)



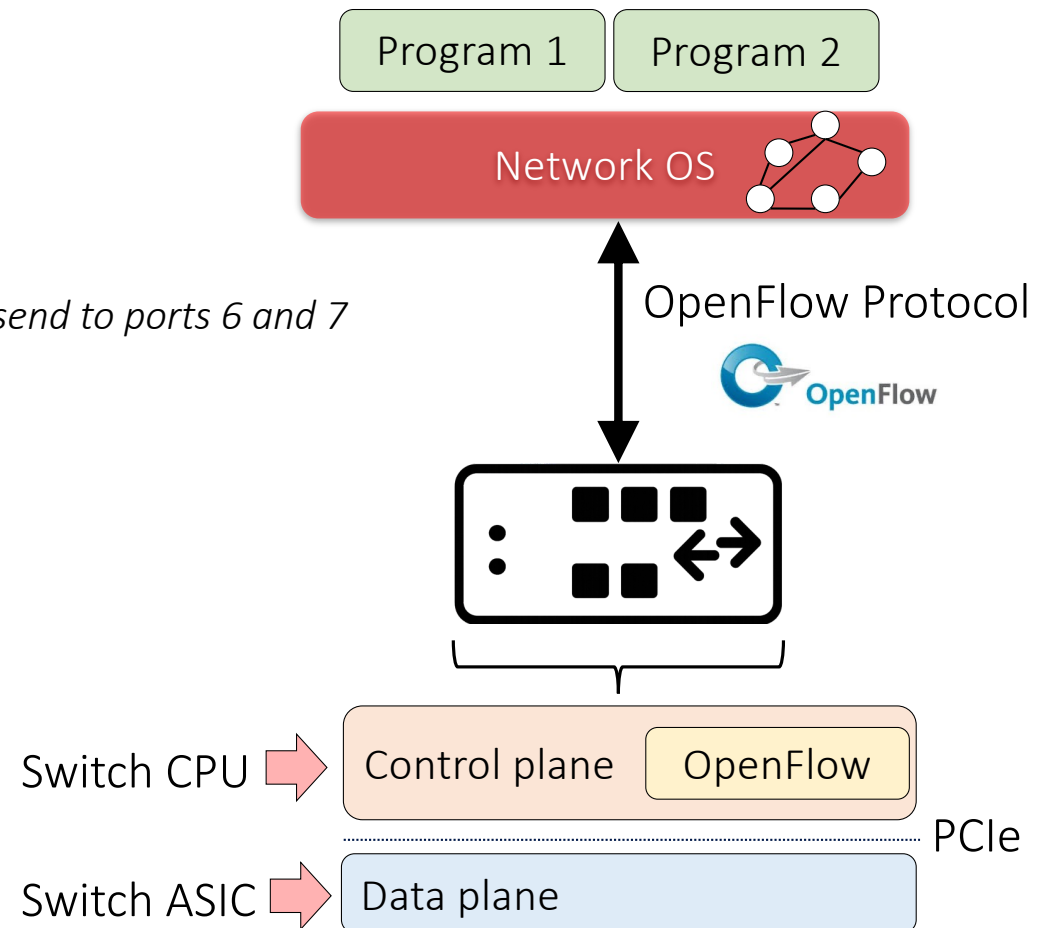
OpenFlow

- OpenFlow is a realisation of SDN
- It is a set of protocols and APIs (controller to switch)



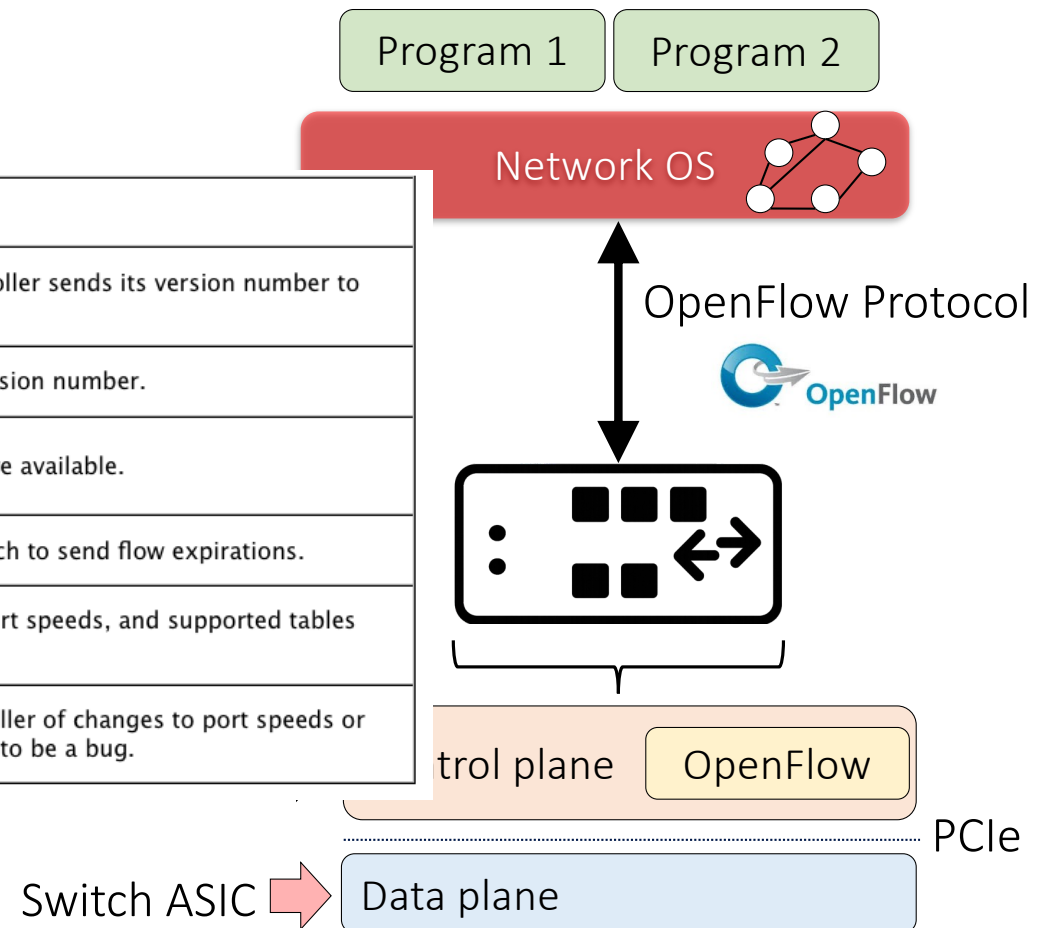
OpenFlow

- Add/remove flow rules at switches
 - *If header = p, then send to port 5*
 - *If header = q, then overwrite header with r and send to ports 6 and 7*
 - *If header = k, then send it to me*
- Statistic collection
- Establish connectivity at switches



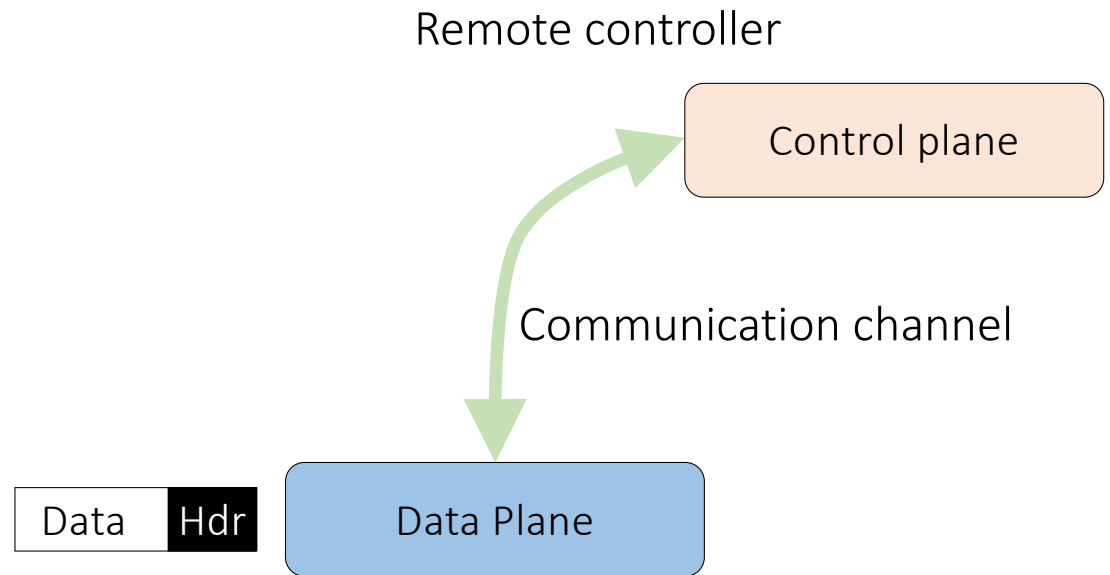
OpenFlow

Message	Type	Description
Hello	Controller->Switch	following the TCP handshake, the controller sends its version number to the switch.
Hello	Switch->Controller	the switch replies with its supported version number.
Features Request	Controller->Switch	the controller asks to see which ports are available.
Set Config	Controller->Switch	in this case, the controller asks the switch to send flow expirations.
Features Reply	Switch->Controller	the switch replies with a list of ports, port speeds, and supported tables and actions.
Port Status	Switch->Controller	enables the switch to inform that controller of changes to port speeds or connectivity. Ignore this one, it appears to be a bug.



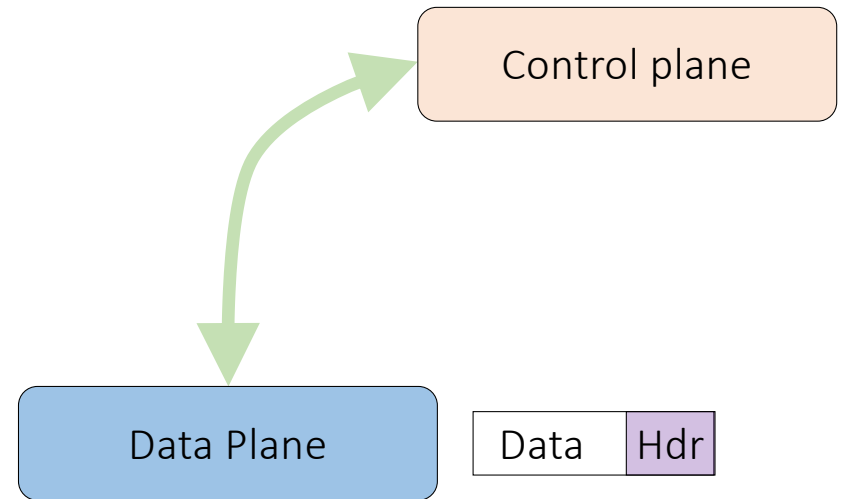
OpenFlow example

- A packet arrives at a switch



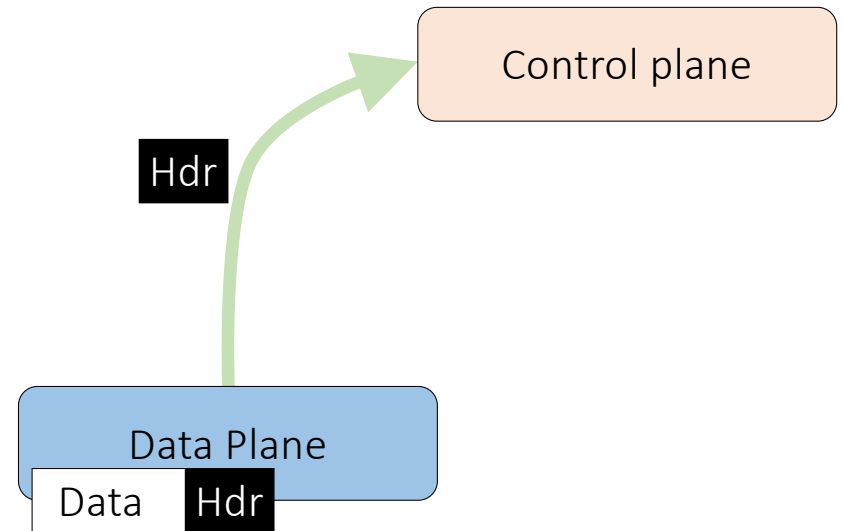
OpenFlow example

- A packet arrives at a switch
- If it matches a rule, it is processed accordingly



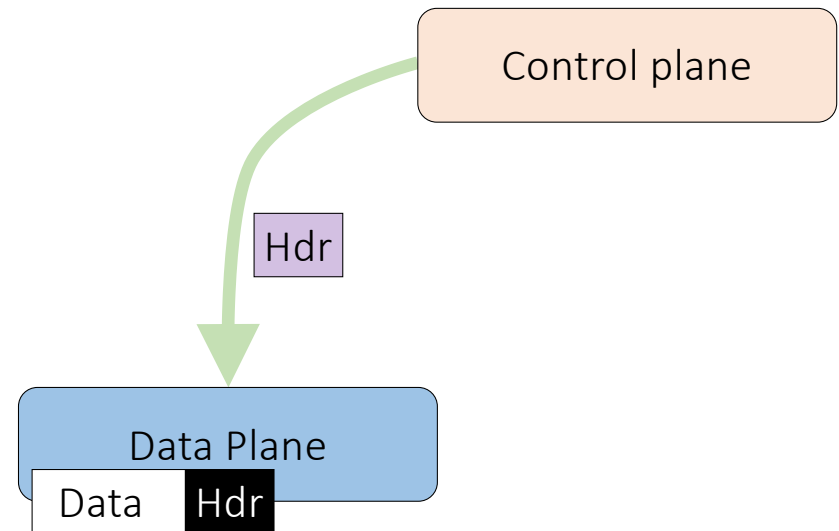
OpenFlow example

- A packet arrives at a switch
- If it does not match any rule, the header is sent to the controller



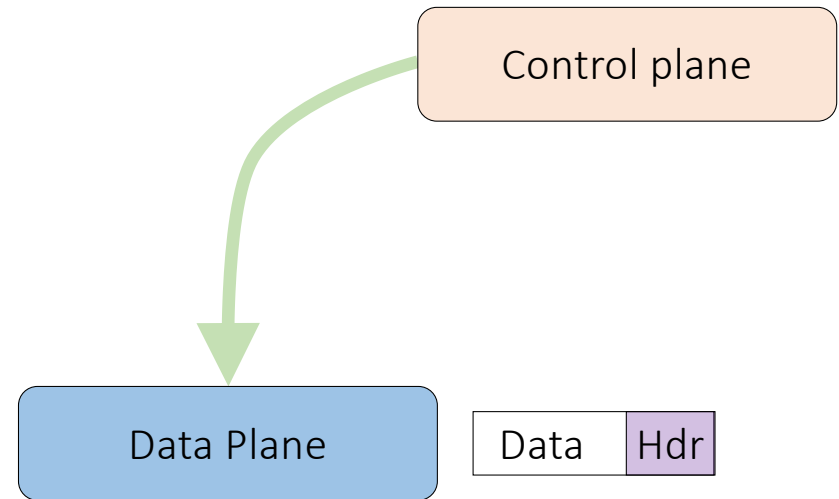
OpenFlow example

- A packet arrives at a switch
- If it does not match any rule, the header is sent to the controller

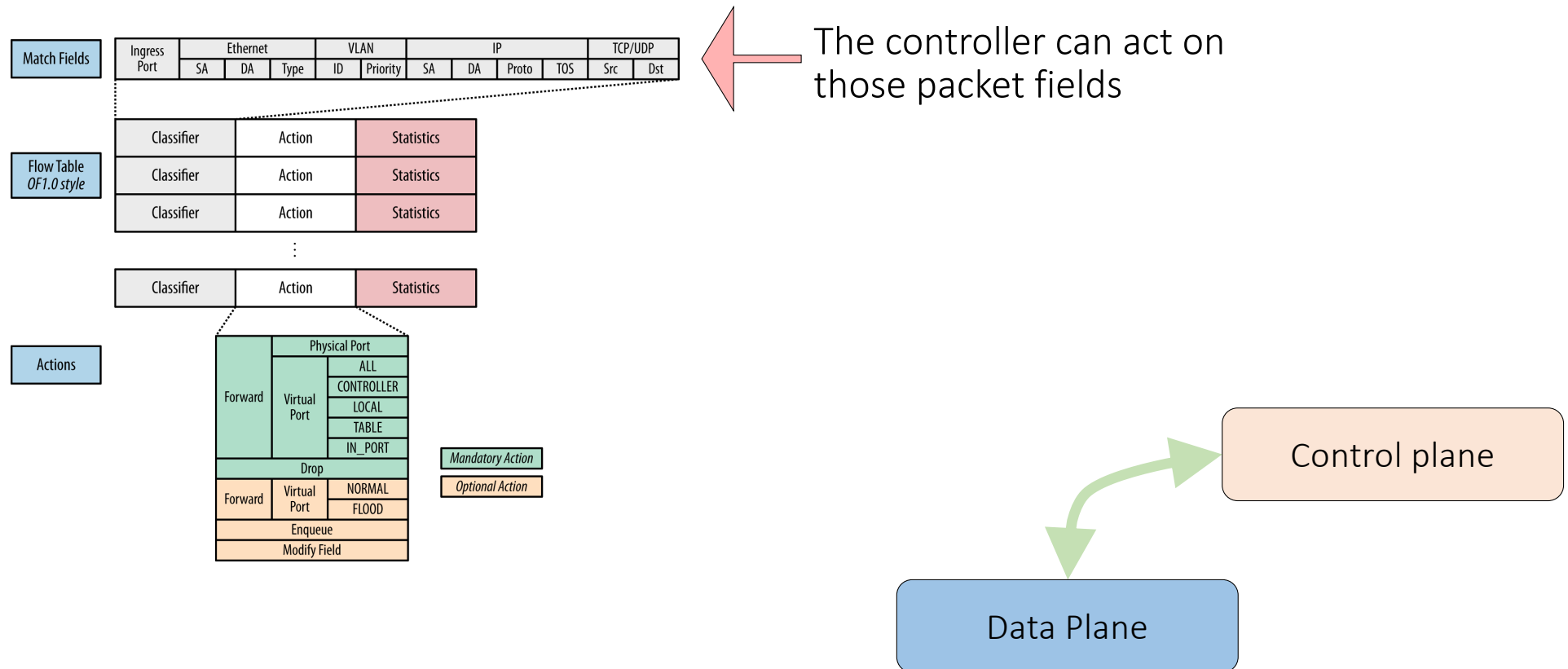


OpenFlow example

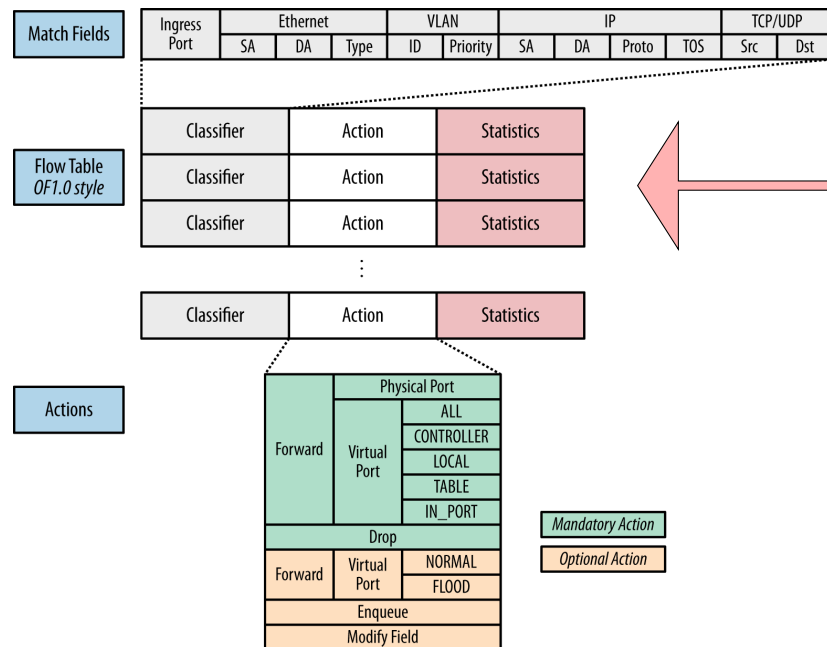
- A packet arrives at a switch
- If it does not match any rule, the header is sent to the controller



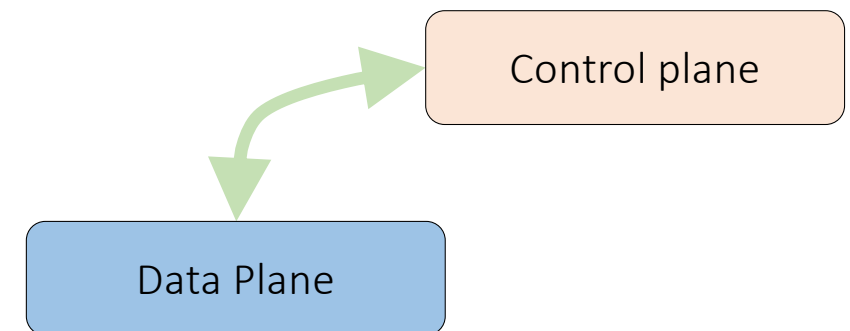
OpenFlow example



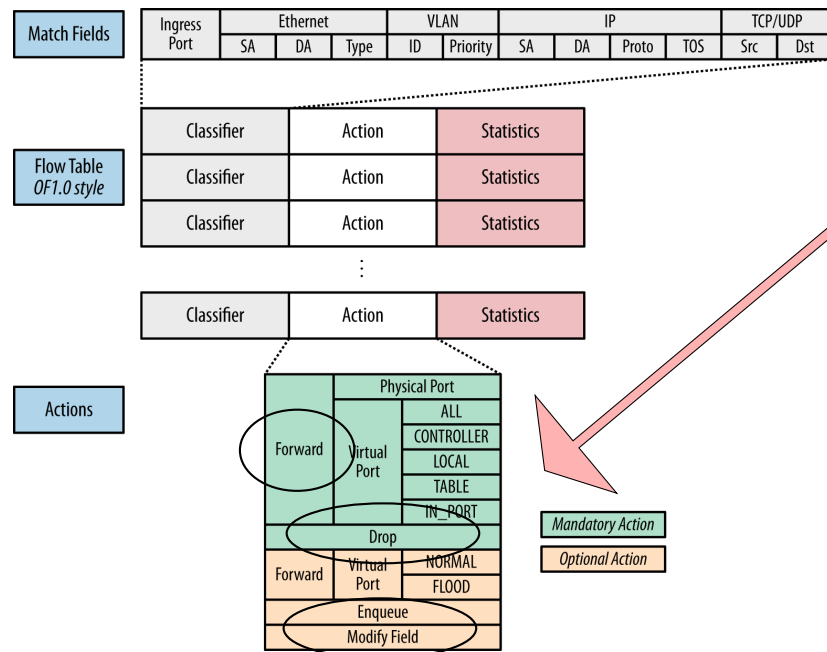
OpenFlow example



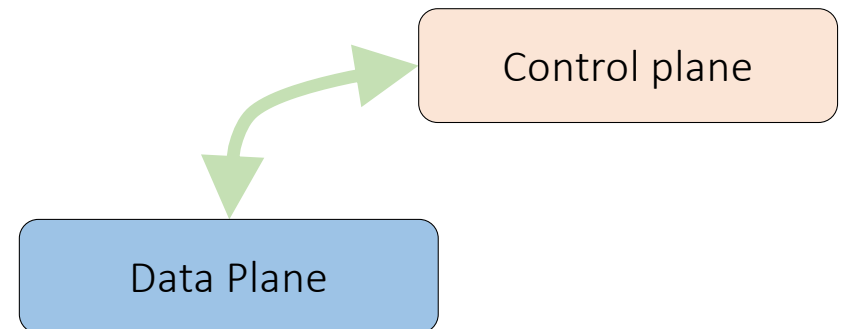
The controller installs the rules in each switch. Rules are a combination of Match + Action. Statistics are also kept in the switch



OpenFlow example

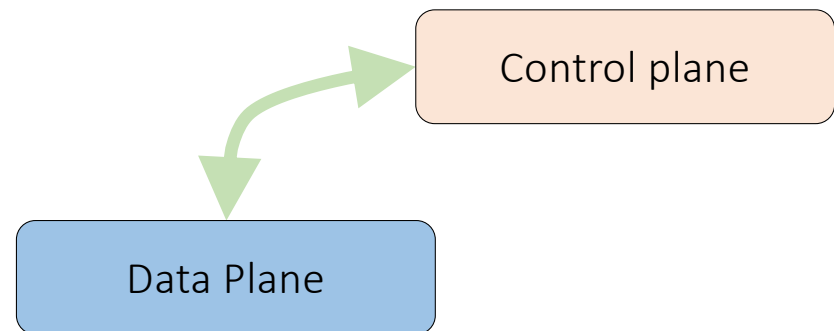


Many types of actions available



OpenFlow modes

- Reactive
- Proactive



OpenFlow modes

Reactive:

- First packet of flow triggers controller to insert flow entry

Proactive:

- Controller pre-populates flow table in switch

OpenFlow modes

Reactive:

- First packet of flow triggers controller to insert flow entry
- Efficient use of flow tables

Proactive:

- Controller pre-populates flow table in switch
- Essentially requires aggregated (wildcard) rules

OpenFlow modes

Reactive:

- First packet of flow triggers controller to insert flow entry
- Efficient use of flow tables
- Every flow incurs small additional flow setup time

Proactive:

- Controller pre-populates flow table in switch
- Essentially requires aggregated (wildcard) rules
- Zero additional flow setup time

OpenFlow modes

Reactive:

- First packet of flow triggers controller to insert flow entry
- Efficient use of flow tables
- Every flow incurs small additional flow setup time
- If controller connection lost, switch has limited utility

Proactive:

- Controller pre-populates flow table in switch
- Essentially requires aggregated (wildcard) rules
- Zero additional flow setup time
- Loss of controller connection does not disrupt traffic

Food for thoughts



- Let's think about OpenFlow...
- Is the OpenFlow data plane a switch or a router?

Food for thoughts



- Let's think about OpenFlow...
- Is the OpenFlow data plane a switch or a router?
- Why OpenFlow enables innovation in the control plane of networks?

Food for thoughts

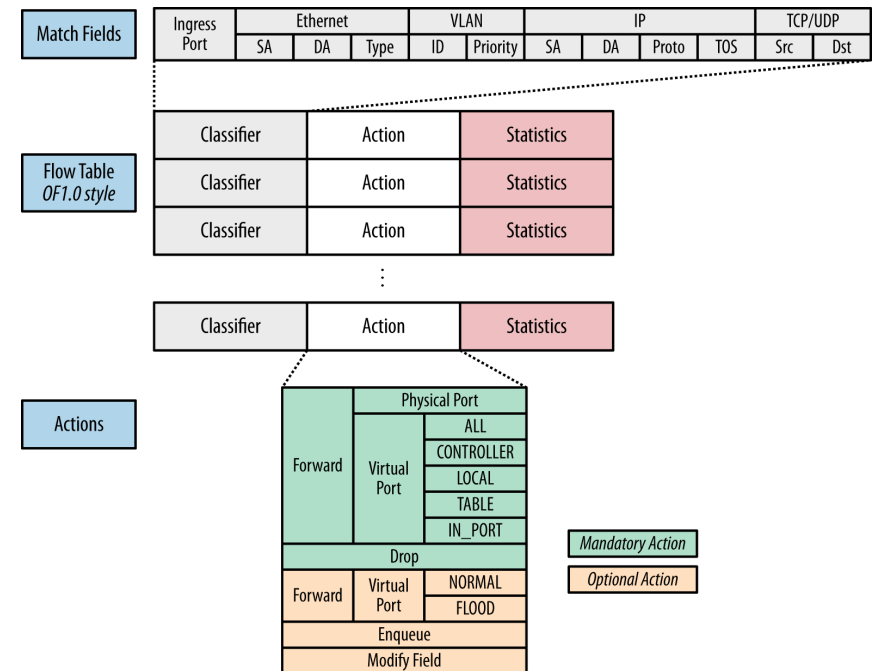
- Let's think about OpenFlow...
- Is the OpenFlow data plane a switch or a router?
- Why OpenFlow enables innovation in the control plane of networks?
- Is it better to run OpenFlow in reactive or proactive mode?

Food for thoughts

- Let's think about OpenFlow...
- Is the OpenFlow data plane a switch or a router?
- Why OpenFlow enables innovation in the control plane of networks?
- Is it better to run OpenFlow in reactive or proactive mode?
- Can we run legacy control planes (i.e., BGP) with OpenFlow?

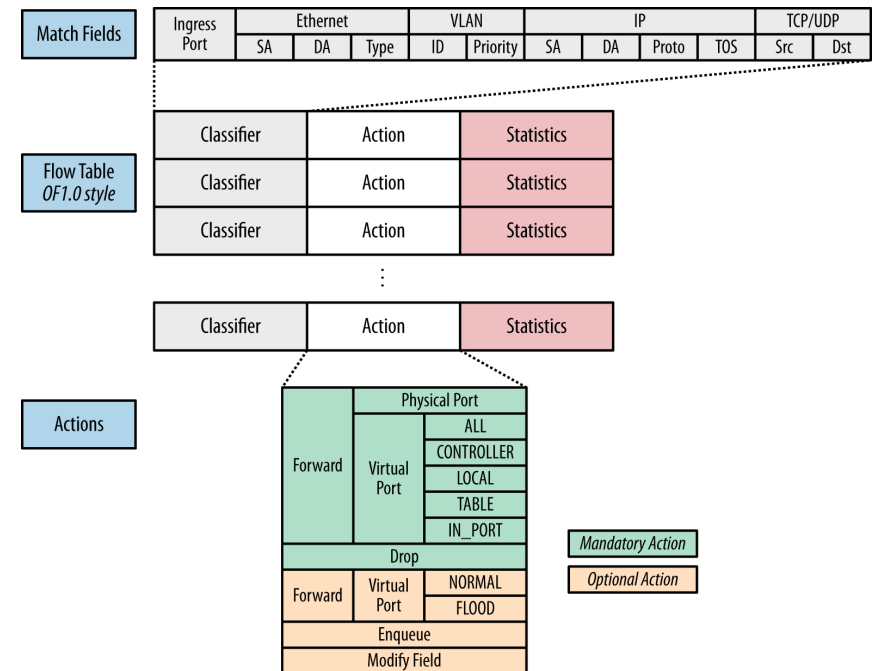
The need for more control

- Back in 2007, when OpenFlow was first conceived the insight was that in most networks switches do pretty much the same things (i.e., Ethernet, IPV4, ACL, VLAN)
- OpenFlow assumes that switches have a fixed, well-known behavior
- You cannot add a new protocol



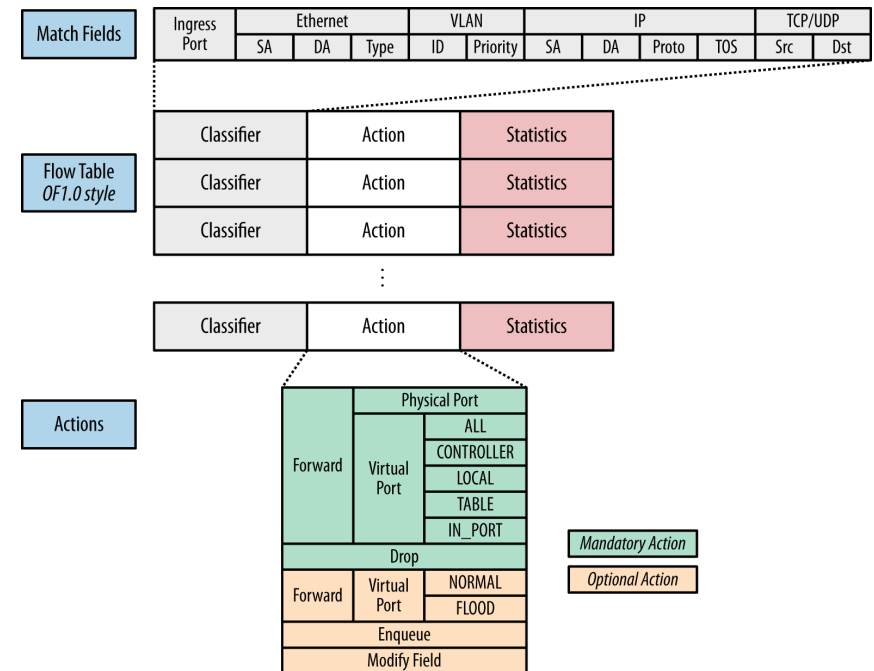
The need for more control

- What if those available are not enough and we seek for more specialised processing?
- This has triggered new OpenFlow versions 1.1, 1.2, 1.3 etc..
- The last version of OpenFlow supports about 50 different header types!!



The need for more control

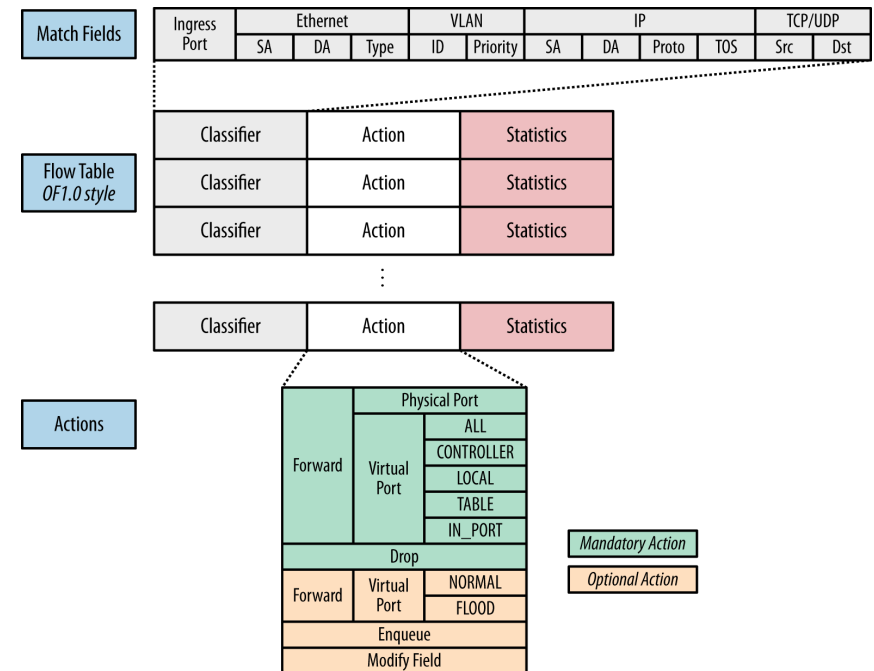
- Everytime we need a new ASIC to support new/more features..



The need for more control

- Everytime we need a new ASIC to support new/more features..

Where is the problem here?

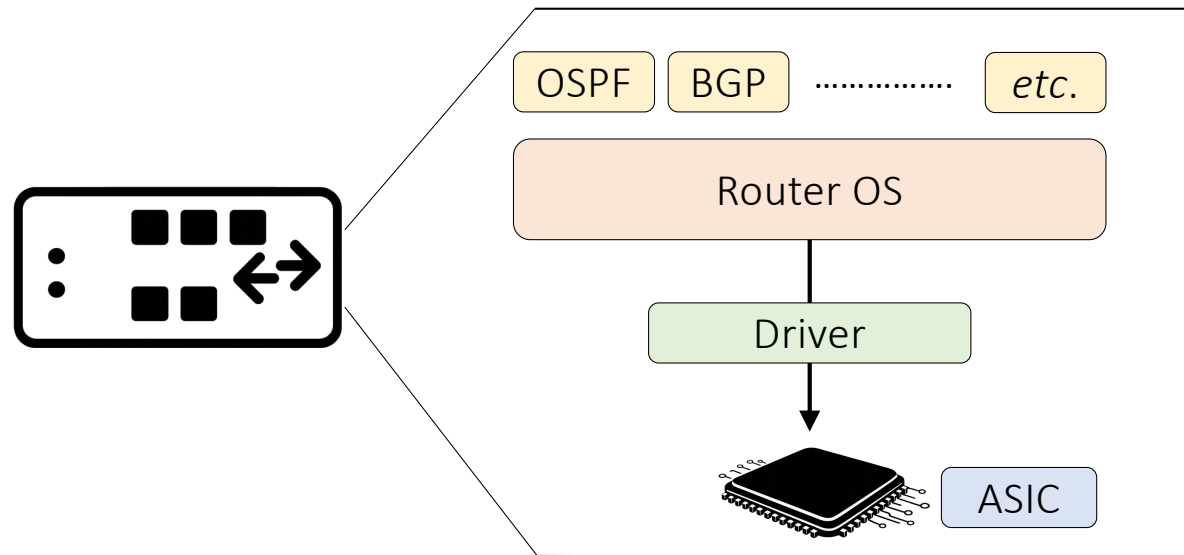


From OpenFlow to...

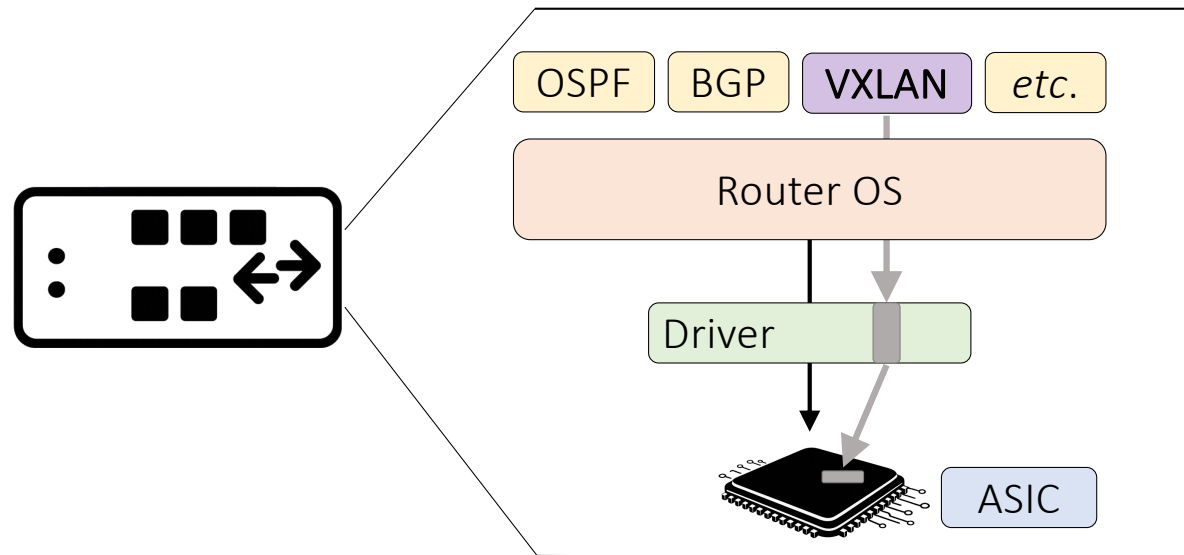


- Instead of repeatedly extending the OpenFlow standard..
- Let's create a flexible mechanism for parsing arbitrary packets and matching (arbitrary) header fields through a common interface

How to add new features to the data plane?

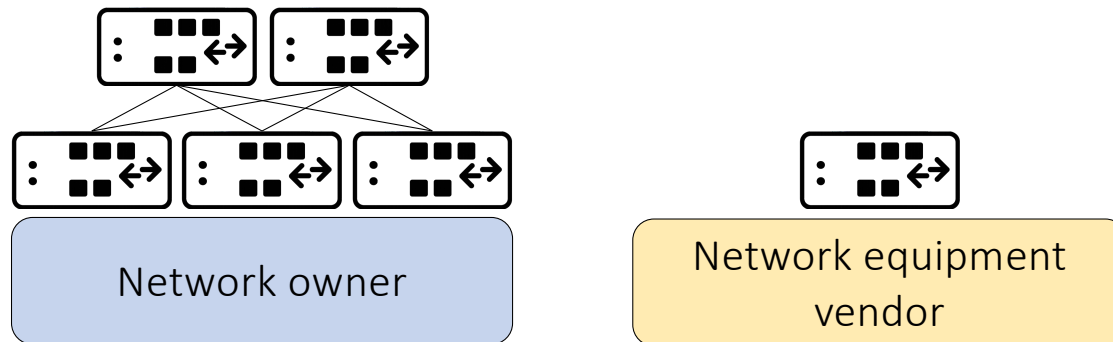


How to add new features to the data plane?

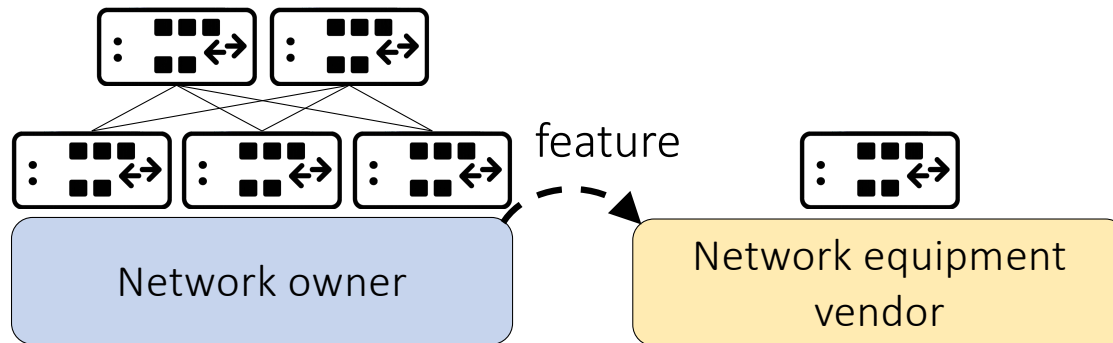


VXLAN (Virtual Extensible LAN) is a network virtualization protocol that encapsulates Ethernet frames inside UDP packets, allowing traffic from different tenants to be identified and isolated in virtualized data centers

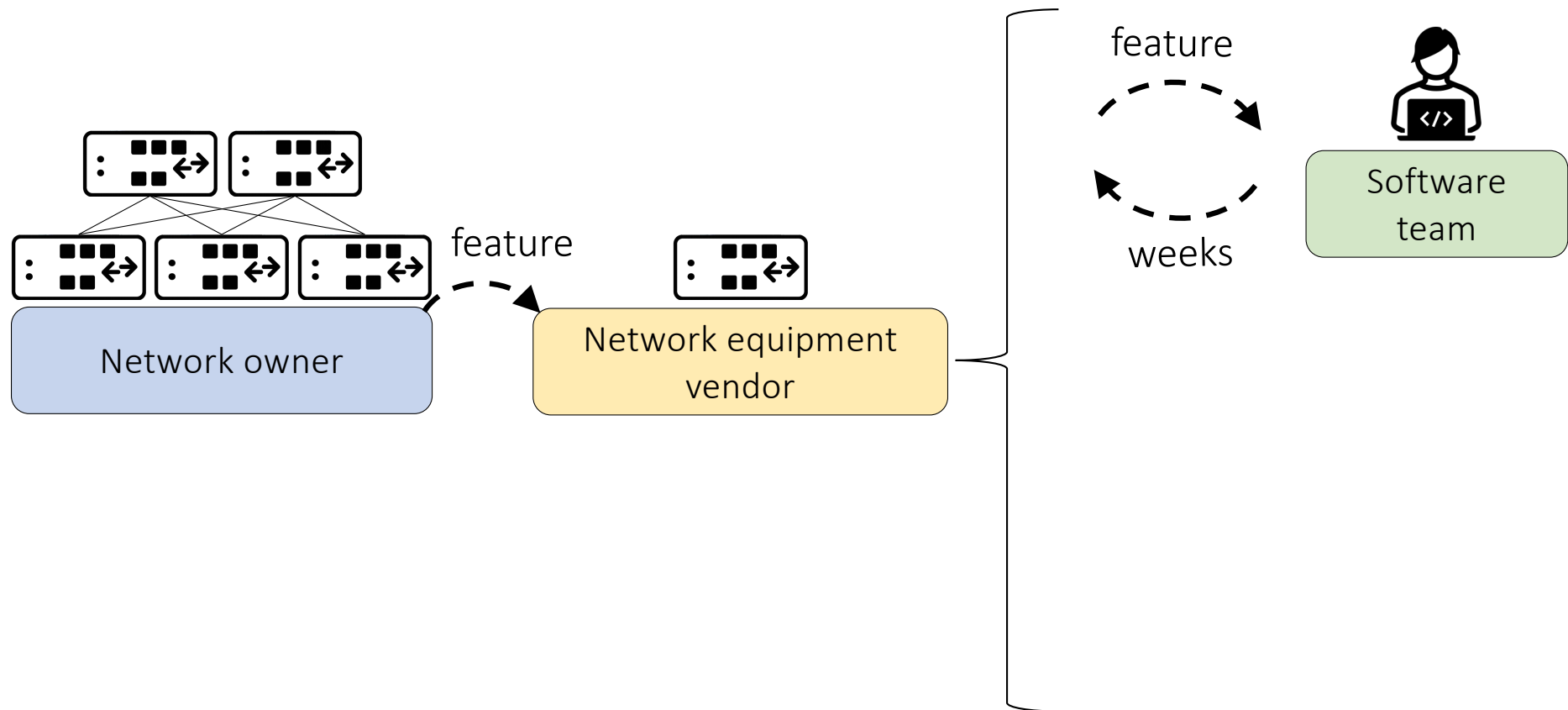
When we need a new feature



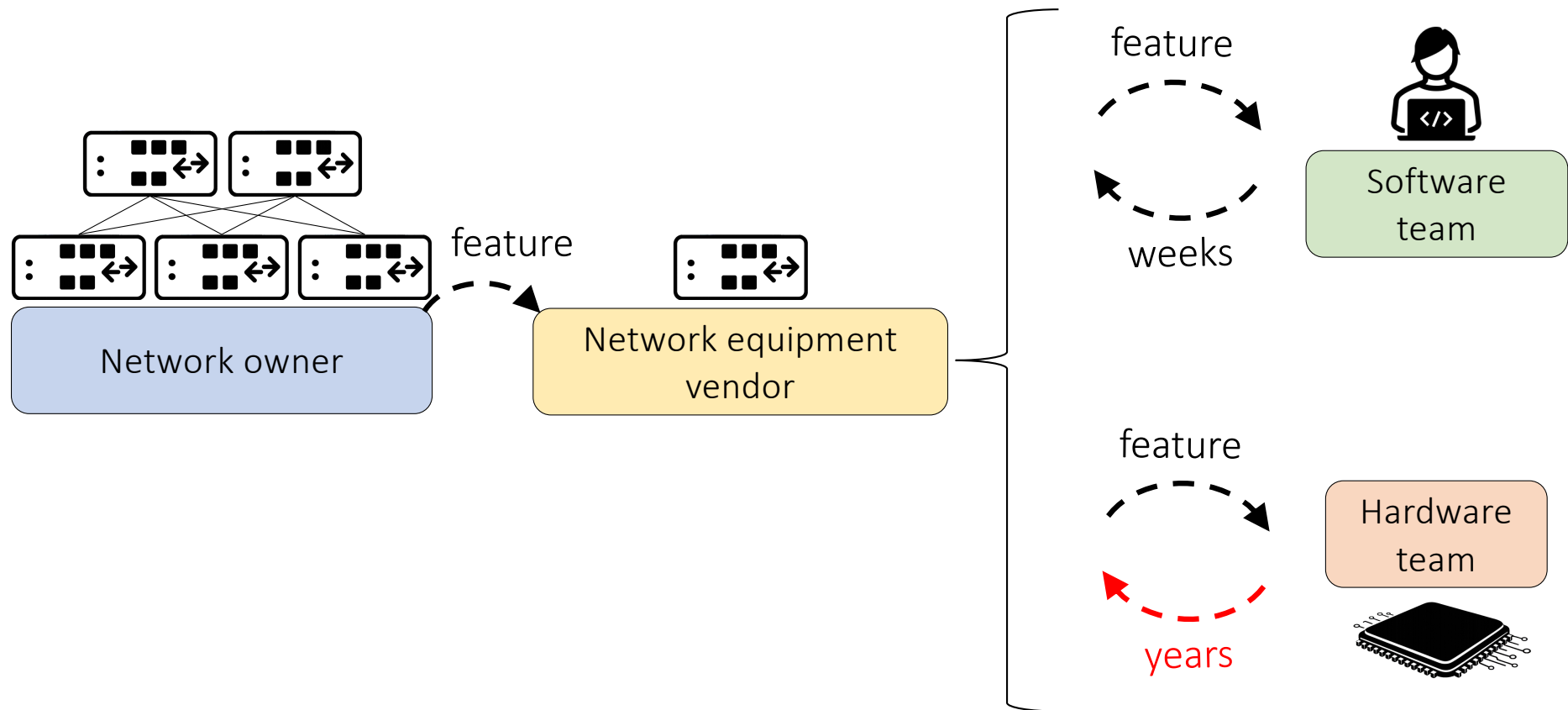
When we need a new feature



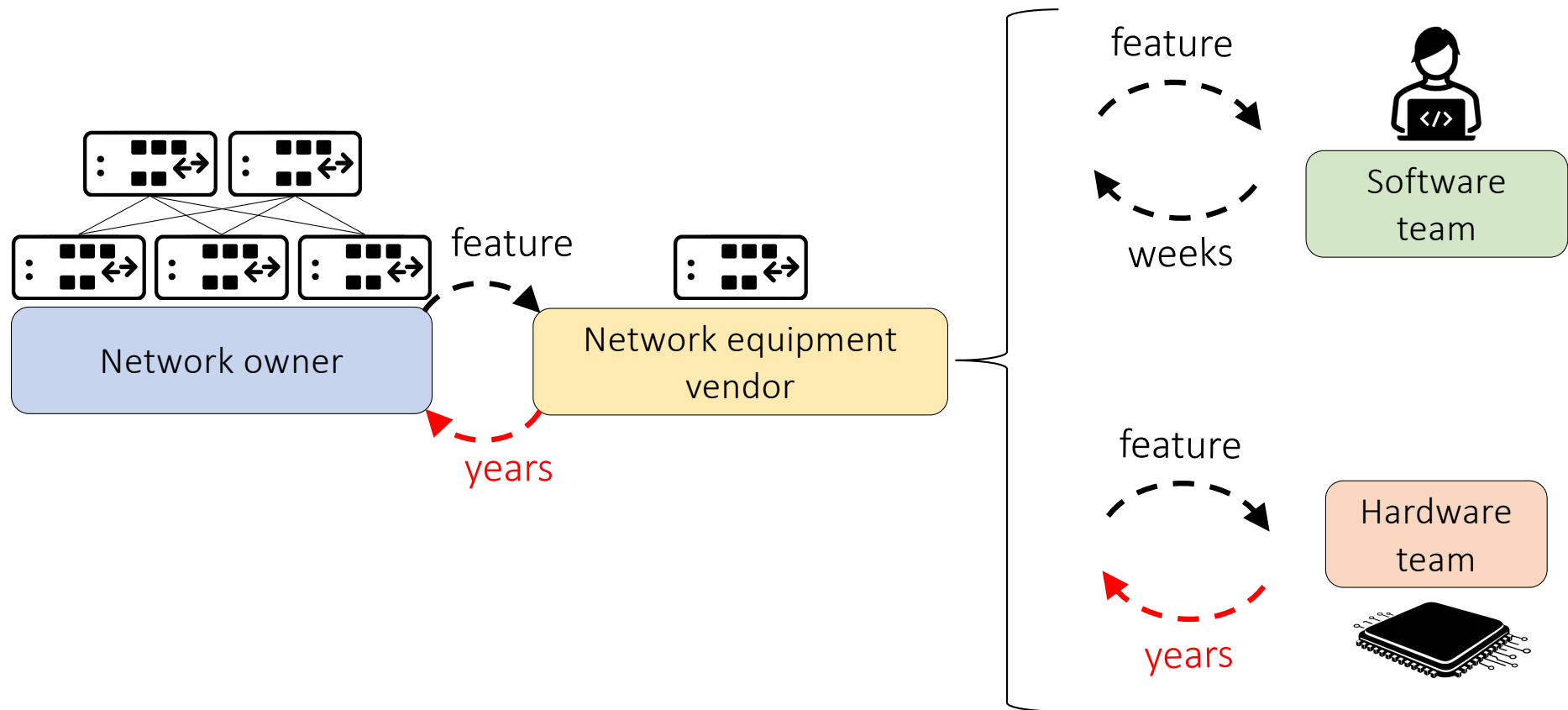
When we need a new feature



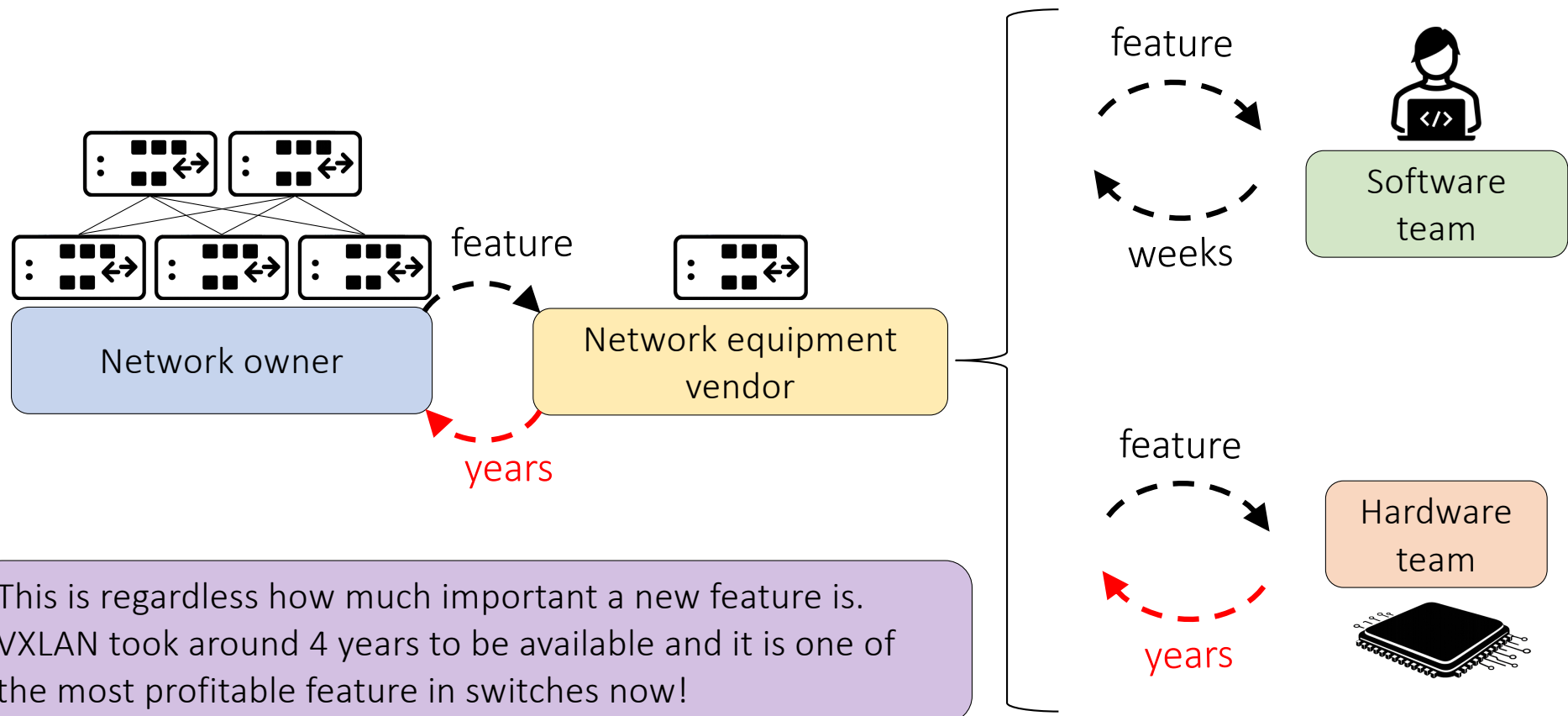
When we need a new feature



When we need a new feature



When we need a new feature



When you need a new data plane feature...

- Equipment vendor **can't** just send you a software upgrade

When you need a new data plane feature...

- Equipment vendor **can't** just send you a software upgrade
- New forwarding features take **years** to develop

When you need a new data plane feature...

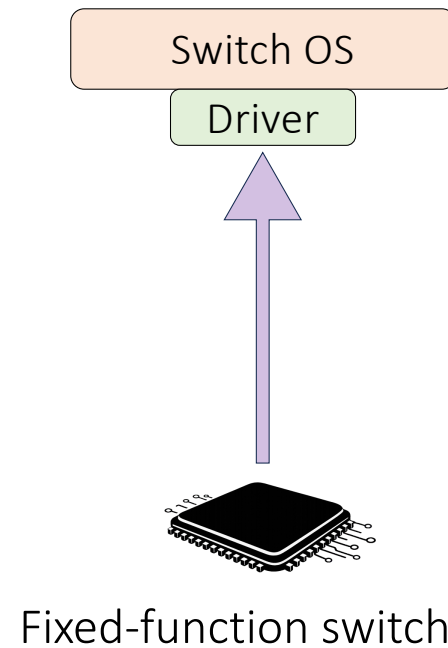
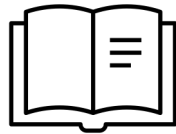
- Equipment vendor **can't** just send you a software upgrade
- New forwarding features take **years** to develop
- By then, you've figured out a kludge to work around it
- Your network gets more complicated

When you need a new data plane feature...

- Equipment vendor **can't** just send you a software upgrade
- New forwarding features take **years** to develop
- By then, you've figured out a kludge to work around it
- Your network gets more complicated
- Eventually, when the upgrade is available, it either
 - No longer solves your problem, or
 - You need a fork-lift upgrade, at huge expense

Network systems are built bottom-up

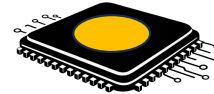
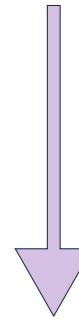
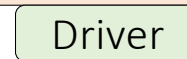
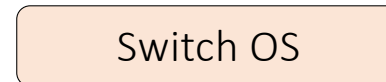
"This is how I process packets ..."



Network systems are built bottom-up

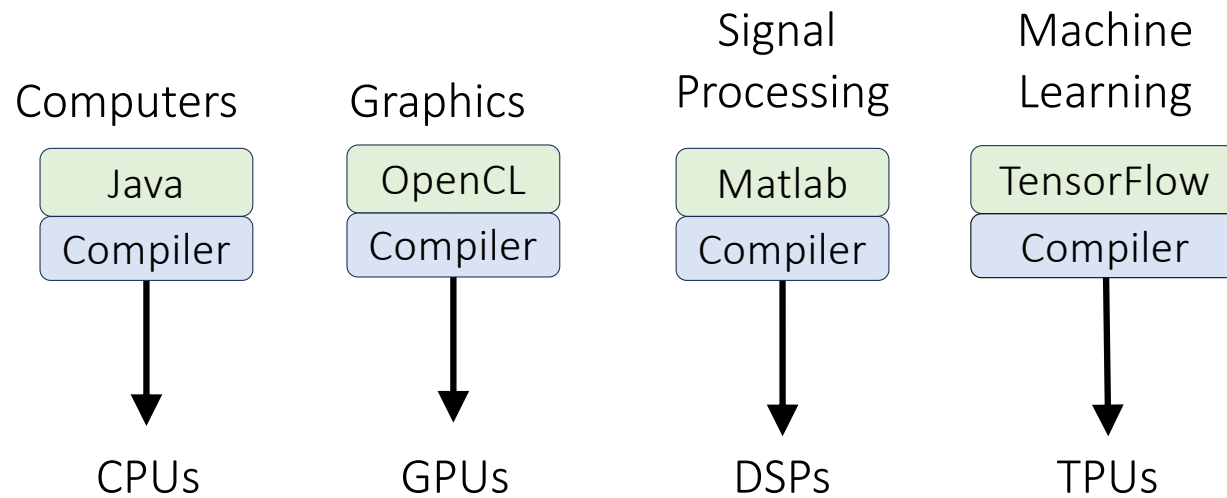


"This is precisely how you must process packets"

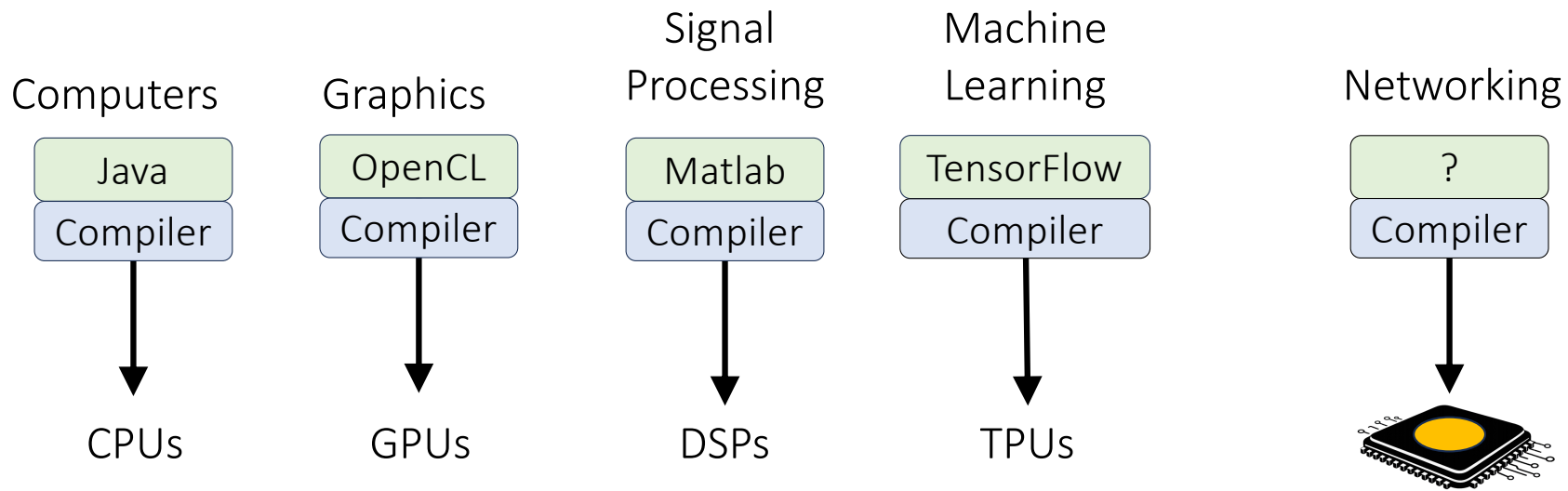


Programmable switch

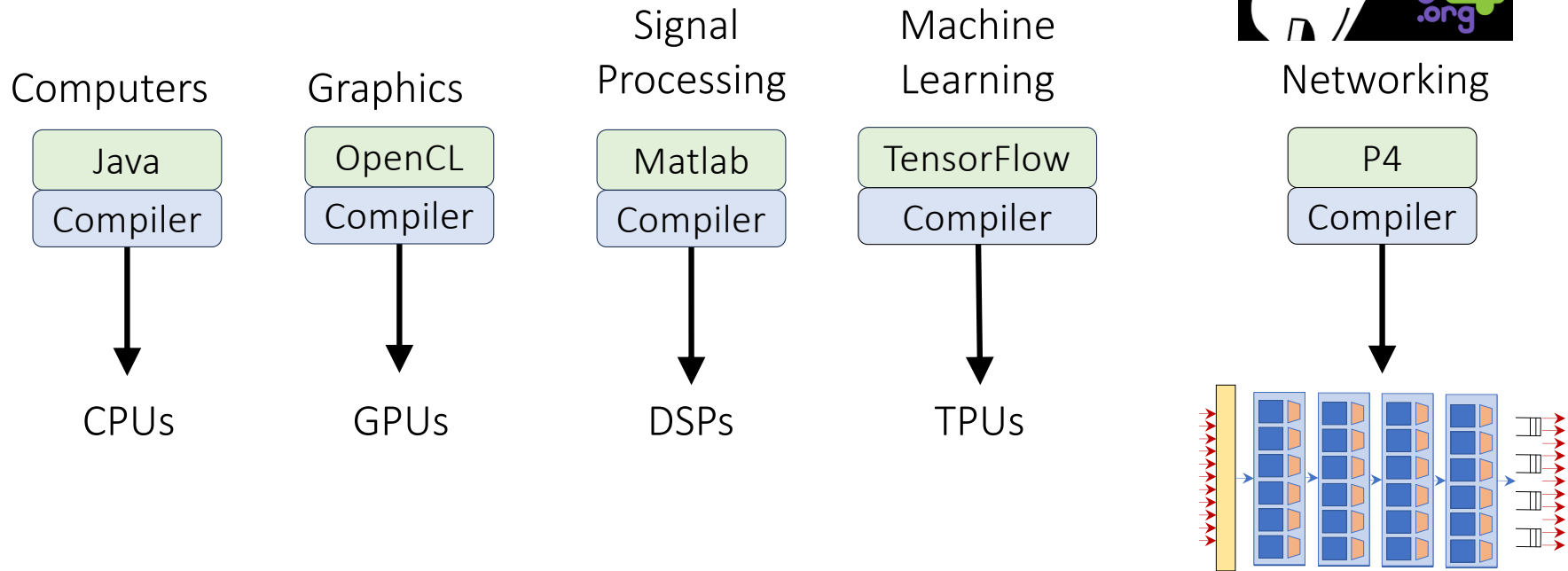
This is not really new generally...



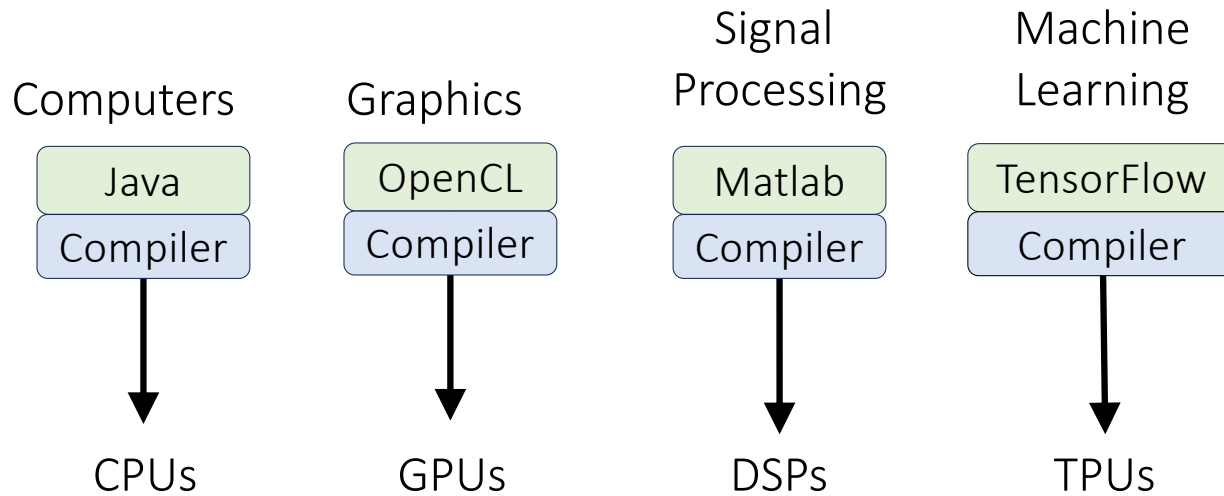
This is not really new generally...



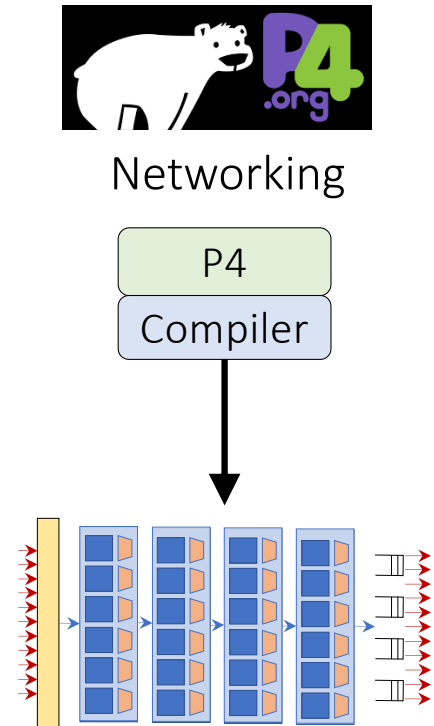
This is not really new generally...



This is not really new generally...



This is matter for the next lecture!

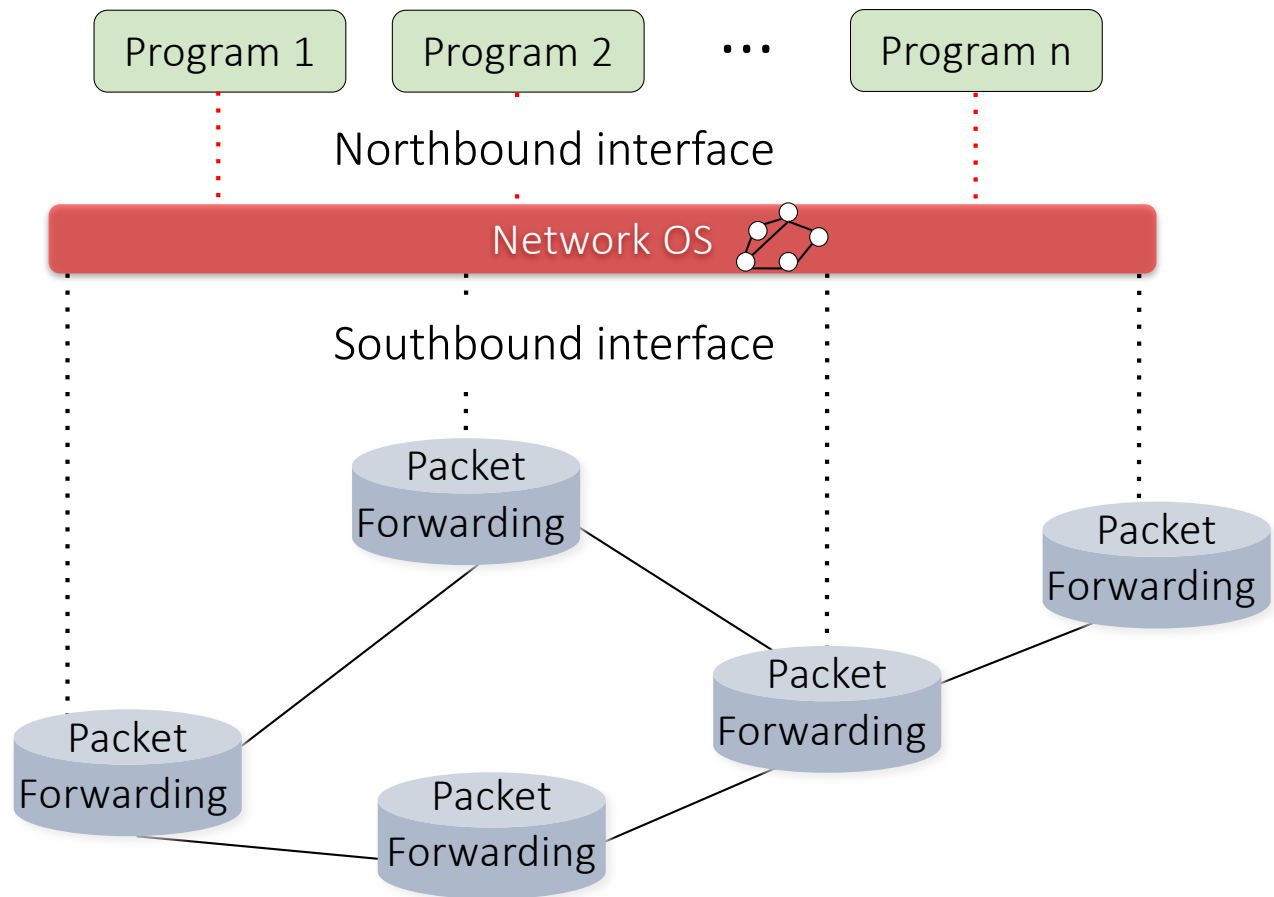


Before we finish..

- SDN has impacted a lot both industry and research
- SDN has enabled industry to customize their networks
- SDN has allowed to easily create new differentiated services
- Most importantly, it has enabled more flexible control and a quicker loop for innovation

SDN impact on research

- New control programs
- New routing algorithms
- Better Network OSes
- Better abstractions



Fun spotlight

Central Control Over Distributed Routing

<http://fibbing.net>

Stefano Vissicchio*, Olivier Tilmans*, Laurent Vanbever†, Jennifer Rexford‡

* Université catholique de Louvain, † ETH Zurich, ‡ Princeton University

* name.surname@uclouvain.be, †ivanbever@ethz.ch, ‡jrex@cs.princeton.edu

ABSTRACT

Centralizing routing decisions offers tremendous flexibility, but sacrifices the robustness of distributed protocols. In this paper, we present *Fibbing*, an architecture that achieves both flexibility and robustness through central control over distributed routing. Fibbing introduces fake nodes and links into an underlying link-state routing protocol, so that routers compute their own forwarding tables based on the augmented topology. Fibbing is expressive, and readily supports flexible load balancing, traffic engineering, and backup routes. Based on high-level forwarding requirements, the Fibbing controller computes a compact augmented topology and injects the fake components through standard routing-protocol messages. Fibbing works with any unmodified routers speaking OSPF. Our experiments also show that it can scale to large networks with many forwarding requirements, introduces minimal overhead, and quickly reacts to network and controller failures.

part of the network. As a network operator, you suspect a denial-of-service attack (DoS), but cannot know for sure without inspecting the traffic as it could also be a flash crowd. Your goal is therefore to: (i) isolate the flows destined to these IP addresses, (ii) direct them to a scrubber connected between *B* and *C*, in order to “clean” them if needed, and (iii) reduce congestion by load-balancing the traffic on unused links, like (*B*, *E*).

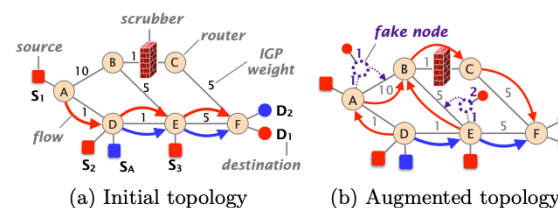


Figure 1: Fibbing can steer the initial forwarding paths (see (a)) for D_1 through a scrubber by adding fake nodes and links (see (b)).